الجمهورية العربية السورية
وزارة التعليم العالي والبحث العلمي
الجامعة السورية الخاصة

Smart Opinion Survey Platform

Junior Project Report

Prepared by:

Yaman Bassam Almatar
Yousef Ahmad Alzoubi
Hasan Abdalruhman Yahya

Supervised by:

Eng. Shadi Blidi
Eng. Maya Alahmad

Project Repository:
https://github.com/Yaman8almatar/smart-survey-platform

Academic Year
2025 – 2026

SUPERVISOR CERTIFICATION

I certify that the preparation of this project entitled **Smart Opinion Survey Platform**, prepared by

Yaman Almatar and Yousef Alzoubi and Hasan Yahya, was made under my supervision at Department of Software & Information System College of Artificial Intelligence Engineering in partial fulfillment of the Requirements for the Degree of Bachelor of Software and Information System Engineering

Name:

Signature:

Date:

Abstract

The Smart Opinion Survey Platform is a web-based intelligent survey system designed to improve the reliability of feedback collection by connecting service providers with demographically qualified respondents. Traditional online survey distribution often depends on public links or uncontrolled sharing, which may lead to random participation, weak respondent qualification, duplicate submissions, and open-text answers that are difficult to analyze consistently. The proposed system addresses these issues through a two-sided platform where service providers create targeted surveys and respondents view only surveys that match their demographic profiles.

The implemented system is built using Django 4.2.11, PostgreSQL 15, Django Templates, Bootstrap, Chart.js, pytest, pytest-django, and Hugging Face Inference API integration. The architecture follows a layered pattern in which views delegate workflows to service classes, services enforce business rules, repositories isolate Django ORM access, domain models define database entities, and the infrastructure layer isolates external AI communication. The system supports service provider registration, respondent registration, authentication, survey creation, question management, targeting criteria, survey publication and closing, response submission, duplicate-response prevention, dashboards, administrative account management, and system reporting.

Open-text answers are processed through a sentiment analysis component that normalizes Hugging Face output into positive, neutral, or negative labels and stores the result state safely. The project demonstrates a complete software engineering lifecycle, including requirements modeling, Agile iterative planning with Sprint 0, UML diagrams, database design, implementation documentation, test planning, traceability, and final evaluation. The final service-layer automated test suite passed successfully with 163 tests and 0 failures, and the GitHub Actions CI workflow completed successfully. The main contribution of the project is the integration of demographic targeting, controlled participation, duplicate-response prevention, and AI-assisted qualitative feedback analysis within one maintainable academic software platform.

Table of Contents

List of Figures

List of Tables

Glossary of Terms and Definitions

Term	Definition
Agile	A software development approach based on incremental delivery, feedback, and adaptation.
Sprint 0	A preliminary Agile phase used for discovery, requirements analysis, architecture planning, environment setup, and technical preparation before implementation.
Use Case	A description of an interaction between an actor and the system to achieve a specific goal.
Functional Requirement	A requirement that defines a function or behavior the system must provide.
Non-Functional Requirement	A requirement that defines a quality attribute such as security, usability, reliability, maintainability, or performance.
Targeting Criteria	Demographic filters used to determine whether a respondent is eligible for a survey.
MVT	Model-View-Template, the architectural structure commonly used by Django applications.
Layered Architecture	An architectural style that separates the system into layers such as presentation, services, repositories, domain models, and infrastructure.
Service Layer	A layer that contains business rules, validation, authorization checks, and workflow orchestration.
Repository Pattern	A design pattern that isolates data access logic from business logic.
ORM	Object-Relational Mapping, a technique that maps application objects to relational database tables.
Django ORM	Django's built-in ORM used to query and persist model objects in the database.
PostgreSQL	The relational database management system used by the project.
Chen ERD	A conceptual entity-relationship notation representing entities, attributes, relationships, and cardinalities.
Relational Schema	A logical database representation using tables, primary keys, foreign keys, and constraints.
Sequence Diagram	A UML diagram that shows object interactions over time.
Activity Diagram	A UML diagram that shows workflow logic, decisions, and process flow.
Class Diagram	a UML diagram that shows system classes, attributes, methods, and relationships.
Sentiment Analysis	An NLP task that classifies text into positive, negative, or neutral sentiment.
Hugging Face Inference API	An external API used by the system to perform sentiment analysis on open-text answers.
RTM	Requirements Traceability Matrix linking requirements to design, implementation, and testing evidence.
SCM	Software Configuration Management, covering version control, branches, releases, and controlled changes.
CI	Continuous Integration, an automated process for running checks and tests after code changes.

Term	Definition
CD	Continuous Delivery or Continuous Deployment. It is not implemented in the current project scope.
GitHub Actions	The CI tool used to run Django checks and service-layer tests automatically.
CSRF	Cross-Site Request Forgery protection used by Django to secure form submissions.
KPI	Key Performance Indicator, a measurable value used to evaluate system performance or project outcomes.

Section One: Project Introduction

Chapter 1: General Introduction

1.1 Project Overview and Background

In digital service environments, decisions about product improvement, customer satisfaction, and operational quality should be based on reliable feedback rather than assumptions. Survey systems are useful only when the collected data represents the intended audience. Public survey links can invite irrelevant participation, uncontrolled samples, and low-confidence results. The Smart Opinion Survey Platform addresses this weakness by combining survey creation with respondent qualification and intelligent analysis.

The platform connects service providers who need reliable feedback with respondents who maintain demographic profiles. A targeting mechanism matches survey criteria against respondent profile data before a survey is displayed. Open-text answers are processed through a sentiment analysis component, transforming qualitative responses into measurable indicators for dashboards and reports.

1.2 Problem Statement and Academic Importance

The core problem is the weak reliability of traditional survey distribution. When a survey is shared publicly, the survey owner cannot ensure that respondents belong to the intended demographic segment. This creates sample bias, reduces decision validity, and may produce misleading recommendations. The second problem is the manual analysis of open-text answers. Manual analysis is slow, inconsistent, and difficult to scale.

From an engineering perspective, the problem is suitable for a senior software project because it requires requirements engineering, role-based workflows, database design, layered architecture, AI service integration, service-level validation, automated testing, and traceability.

1.3 Project Objectives (Main objective , Sub-Objectives)

The main objective of this project is to design and implement a smart two-sided web-based survey platform that connects service providers with demographically matched respondents. The platform aims to improve the reliability of survey participation by reducing random access to surveys and by supporting decision-making through automated analysis of open-text responses. The project objectives are defined in a measurable form to ensure that each objective can be validated during implementation, testing, and final evaluation.

Table 1.1: Project Objectives

Objective ID	Objective	Measurable Outcome
OBJ-M	Build a two-sided survey platform connecting service providers and respondents.	The system provides separate workflows for service providers, respondents, and administrators, including registration, authentication, role-based navigation, and role-specific dashboards.
S-OBJ-01	Reduce random survey participation through demographic targeting and eligibility filtering.	Respondents can view only published surveys that match their demographic profiles and have not been previously answered by them.
S-OBJ-02	Automate the interpretation of open-text answers using sentiment analysis.	Open-text answers are processed through the sentiment analysis component and stored with one of the following analysis states: positive, neutral, negative, or failed.
S-OBJ-03	Provide dashboards and reports to support decision-making.	Service providers can view survey-level analysis results, while administrators can view aggregate platform metrics related to users, surveys, and responses.
S-OBJ-04	Apply disciplined software engineering practices throughout the project lifecycle.	The project documentation includes requirements analysis, UML diagram placeholders, architectural design, database design, implementation structure, testing evidence, and requirements traceability.

1.4 Project Scope, Constraints, and Assumptions

This section defines the boundaries of the Smart Opinion Survey Platform. The project scope identifies the functions that are included in the current implementation, while the out-of-scope items clarify what is intentionally excluded from this version. The constraints describe the limitations that affected the project, and the assumptions define the conditions expected to be true for the system to operate correctly.

Table 1.2: Project Scope, Constraints, and Assumptions

Category	Description
In Scope	The system includes user registration, login and logout, demographic profile management, survey creation, survey updating, survey deletion, question management, targeting criteria definition, survey publication and closing, eligible survey listing, response submission, duplicate response prevention, sentiment analysis for open-text answers, provider dashboards, administrator account management, and system reports.
Technical Scope	The system is implemented as a Django 4.2.11 web application using PostgreSQL 15, Django ORM, Django Templates, Bootstrap, Bootstrap Icons, Chart.js, pytest, pytest-django, requests, and Hugging Face Inference API integration.
Out of Scope	The current version does not include training a machine learning model from scratch, a native mobile application, a public REST API, payment or reward mechanisms, advanced respondent quality scoring, asynchronous AI processing, or full production-scale deployment.
Constraints	The project is constrained by the limited semester duration, limited infrastructure resources, dependency on the availability and response time of the external Hugging Face Inference API, and the need to insert final diagrams, screenshots, and test execution evidence before final submission.
Assumptions	The system assumes that respondents provide accurate demographic data, service providers define valid targeting criteria, environment variables are configured correctly, PostgreSQL is available during execution, and final screenshots and diagrams will be inserted into the report before submission.

1.5 Contribution to Knowledge and Field

The project contributes an academic implementation that integrates demographic targeting and AI-assisted feedback analysis inside one maintainable web platform. The innovation is not the invention of a new machine learning model. The contribution is the engineering integration of controlled eligibility, duplicate prevention, qualitative feedback processing, dashboards, and traceability into a coherent software system.

1.6 Research Methodology and Scientific Methods

This report follows an **engineering research methodology**. The project is handled as a software engineering problem that requires problem analysis, requirements identification, system design, implementation, testing, and evaluation. This methodology is suitable because the project aims to build a working smart software system, not to propose a purely theoretical algorithm. The problem is analyzed from two main perspectives. The first is **data reliability**, where the project addresses the weakness of random survey participation by using demographic targeting and eligibility filtering. The second is **decision support**, where the system applies sentiment analysis to open-text answers and converts them into structured indicators that can support dashboards and reports.

The scientific methods used in this project include stakeholder analysis, use case modeling, requirements analysis, architectural design, database modeling, implementation-based validation, and test case mapping. The requirements are derived from the needs of service providers, respondents, administrators, academic supervisors, and future maintainers. These requirements are then translated into architecture components, database entities, implementation modules, and testing evidence.

This methodology ensures that each major requirement is connected to the corresponding analysis, design, implementation, and testing elements. Therefore, the project can be evaluated as a complete engineered system rather than as a set of isolated implementation files.

1.7 Document Organization

The document is organized according to the university senior project guide. Section One introduces the project and development methodology. Section Two presents the theoretical study and requirements analysis. Section Three documents design and architecture. Section Four documents implementation and testing. Section Five presents results, analysis, conclusions, and recommendations. Section Six contains appendices and references.

1.8 Basic Definitions and Concepts

The following terms represent the basic concepts used throughout the analysis, design, implementation, and testing chapters of this report.

Table 1.3: Basic Definitions and Concepts

Term	Definition
Two-Sided Survey Platform	A platform that connects service providers who create surveys with respondents who answer them.
Demographic Targeting	A filtering method that matches surveys with respondents based on age, gender, and region.
Eligibility Filtering	A process that shows respondents only the surveys they are allowed to answer.
Sentiment Analysis	An NLP technique that classifies open-text answers as positive, neutral, or negative.
Layered Architecture	An architecture style that separates the system into presentation, service, repository, data, and infrastructure layers.
Service Layer	The layer responsible for business rules, validation, authorization, and workflow control.
Repository Pattern	A design pattern that separates database access from business logic.
Relational Schema	A database design that represents entities as tables with keys and constraints.
SCM	Software Configuration Management controls source code versions, branches, and releases.
Requirements Traceability	A method for linking requirements to use cases, design, implementation, and tests.

Chapter 2: Project Development and Management Methodology

2.1 Selected Development Methodology and Justification

The project was managed using an **Agile Iterative and Incremental Development** methodology. This methodology is suitable because the system consists of several independent but connected modules, including authentication, survey management, demographic targeting, response submission, sentiment analysis, dashboards, and administration. These modules can be planned, implemented, reviewed, and improved across multiple sprints.

A preliminary **Sprint 0** was added before implementation. Its purpose was to define the project scope, analyze requirements, prepare the MVP, organize the product backlog, prioritize use cases, prepare the initial SRS, create the core UML placeholders, and establish the architecture baseline. This step was necessary because starting implementation before requirements analysis and design would violate disciplined software engineering practice.

This methodology supports the project by allowing the team to deliver the platform gradually while maintaining traceability between requirements, design, implementation, and testing. It also reduces development risk because each sprint produces a clear output that can be reviewed before moving to the next stage.

2.2 Project Schedule and Sprint Plan

The project schedule was organized as an Agile iterative execution plan from 1 March 2026 to 11 May 2026. Sprint 0 was included before implementation to cover requirements analysis, MVP scope definition, UML preparation, architecture baseline, ERD, and implementation readiness. Figure 2.1 presents the approved sprint-based Gantt chart.

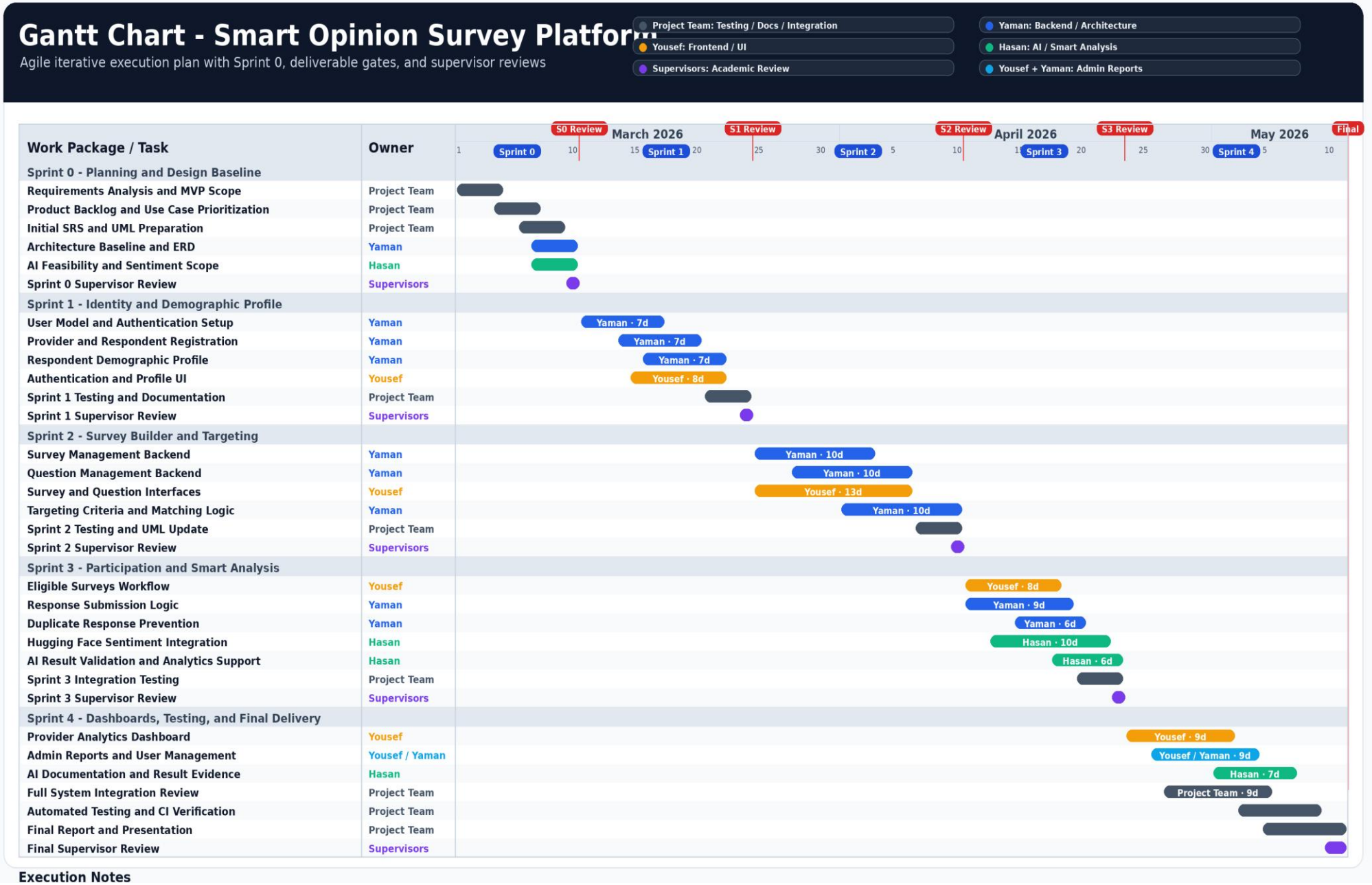

Figure 2.1: Agile Gantt Chart

Table 2.1: Sprint-Based Project Execution Plan

Sprint	Period	Main Objective	Key Deliverables
Sprint 0	Mar 1 – Mar 10, 2026	Requirements, planning, and system design preparation.	MVP scope, backlog, use cases, initial SRS, UML placeholders, architecture baseline, and ERD baseline.
Sprint 1	Mar 11 – Mar 24, 2026	Identity management and demographic profile implementation.	Registration, login, role-based access, respondent profile, and initial database schema.
Sprint 2	Mar 25 – Apr 10, 2026	Survey builder and targeting engine implementation.	Survey management, question management, targeting criteria, and matching logic.
Sprint 3	Apr 11 – Apr 23, 2026	Participation workflow and smart analysis integration.	Eligible surveys, response submission, duplicate prevention, and sentiment analysis.
Sprint 4	Apr 24 – May 11, 2026	Results, integration, testing, and academic delivery.	Dashboards, full integration, final testing evidence, report, and presentation.

2.3 Risk Management

Table 2.2: Risk Register

Risk ID	Risk	Probability	Impact	Mitigation
R-01	External AI API becomes unavailable.	Medium	High	Store the response first, mark sentiment analysis as **FAILED**, and allow future retry.
R-02	Incorrect targeting criteria are defined.	Medium	High	Validate gender, age range, and region through **Targeting Service** before saving criteria.
R-03	Duplicate respondent submissions occur.	Low	High	Apply service-level duplicate checks and enforce a database unique constraint.
R-04	Separation of concerns is violated.	Medium	High	Preserve the dependency direction: **views → services → repositories → ORM**.
R-05	Test execution evidence is insufficient.	Medium	Medium	Run **pytest** in a valid Python environment and attach the execution screenshot.
R-06	Final diagrams are inserted late.	Medium	Medium	Reserve diagram placeholders and insert final UML, ERD, architecture, and testing diagrams before submission.
R-07	Git branch divergence affects final delivery.	Medium	Medium	Synchronize **develop** with **origin/develop** and document the final branch status before submission.

2.4 Team Roles and Responsibilities

Table 2.3: Team Roles and Responsibilities

Team Member / Role	Responsibility Area	Main Tasks
YAMAN	Backend Development and Architecture	Django project structure, domain models, service layer, repository layer, database design, and architecture consistency.
YOUSEF	Frontend and User Interface	Django templates, forms, Bootstrap pages, provider and respondent workflows, dashboard presentation, and UI screenshots.
HASAN	AI Integration and Smart Analysis	Hugging Face integration, sentiment analysis validation, analytics support, and intelligent component documentation.
Project Team	Quality Assurance and Documentation	Testing support, UML updates, report preparation, integration review, and presentation materials.
Supervisors	Academic Supervision	Reviewing methodology, requirements quality, engineering design, documentation structure, and final deliverables.

2.5 Quality Management

Quality management was applied throughout the sprints rather than postponed until the final stage. The project quality controls include layered architecture, service-level validation, repository isolation, database constraints, automated test cases, documentation, and requirements traceability.

The repository includes **160 service-layer tests** covering authentication, profile management, survey lifecycle, question management, targeting, response submission, sentiment analysis, and administration. The test suite was executed successfully using **pytest**, and the final execution evidence is attached in **Appendix C**. This confirms that the main business rules were validated before final delivery.

2.6 Project Management Tools

The project used **Git** and **GitHub** for source control and collaboration. The repository also includes a **docs** folder for architecture and UML documentation, **pytest** and **pytest-django** for automated testing, **Docker Compose** for PostgreSQL setup, and environment variables for configuration and secret management.

These tools supported version control, documentation organization, automated testing, database setup, and environment-specific configuration during development.

2.7 Monitoring and Continuous Evaluation

Project monitoring was performed through weekly progress reviews, sprint deliverable checks, implementation review, test execution, and continuous documentation updates. The team also conducted periodic reviews with the academic supervisors to validate the project direction, requirements quality, architectural decisions, and documentation structure.

Continuous evaluation was based on comparing the implemented modules with the planned requirements, use cases, and test cases. Supervisor feedback was used to refine the report structure, improve the engineering documentation, and ensure that the final deliverables remained aligned with the academic project requirements.

2.8 Software Configuration Management

Software Configuration Management (SCM) was applied to control source code changes, branch organization, version history, dependencies, database migrations, and environment configuration. The project source code was managed using **Git** and hosted on **GitHub** under the repository **Yaman8almatar/smart-survey-platform**.

The repository contains the complete Django web application, including application modules, configuration files, domain models, services, repositories, infrastructure integration, templates, static files, tests, documentation, dependency files, and Docker configuration. This structure supports controlled development and improves traceability between requirements, implementation, testing, and final delivery.

2.8.1 Git Repository Structure

The project was maintained in a single GitHub repository because the system is implemented as one Django web application rather than separated frontend and backend repositories. The repository includes the main application code, documentation, test files, configuration files, and dependency management files.

Table 2.4: Git Repository Structure

Repository Element	Purpose
apps/	Contains Django applications for users, surveys, responses, analysis, admin panel, and core models.
services/	Contains business logic, validation rules, authorization checks, and workflow orchestration.
repositories/	Contains database access logic using Django ORM.
infrastructure/	Contains external integration code, including the Hugging Face sentiment analysis client.
templates/	Contains shared Django templates and layout files.
static/	Contains CSS, JavaScript, and static frontend assets.
tests/	Contains automated tests for service-layer behavior and business rules.
docs/	Contains architecture, requirements, UML, and implementation documentation.
requirements.txt	Defines Python package dependencies.
docker-compose.yml	Provides PostgreSQL setup for the development environment.
.env.example	Documents required environment variables without exposing secrets.

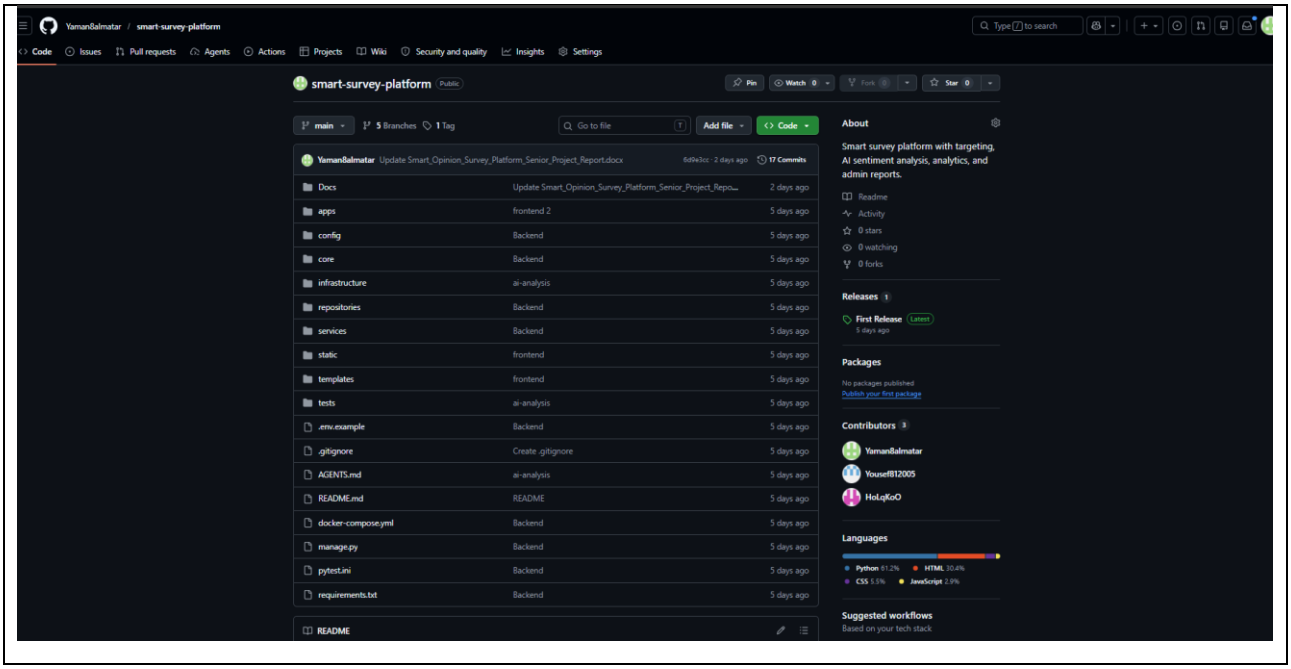

Figure 2.2: GitHub Repository Overview

2.8.2 Branching and Merging Strategy

A simplified branching strategy was adopted to support Agile incremental development and controlled integration. The repository uses a stable **main** branch, an integration **develop** branch, and feature branches for isolated development work.

The **main** branch represents the stable integrated version of the system. The **develop** branch is used to combine completed features before final merging. Feature branches are created for specific development areas such as backend core implementation, frontend implementation, and AI analysis.

Table 2.5: Branching Strategy

Branch	Purpose
main	Stable branch used for final integrated delivery.
develop	Integration branch used to combine completed feature work before release.
feature/backend-core	Feature branch used for backend core implementation.
feature/frontend	Feature branch used for frontend and user interface implementation.
feature/ai-analysis	Feature branch used for AI and sentiment analysis implementation.

This branching structure improved development isolation, reduced uncontrolled changes, and allowed each major feature area to be reviewed before integration.

Figure 2.3: SCM Branching Strategy Screenshot

2.8.3 Development Workflow

The development workflow followed an incremental and task-based process. Each major task was implemented in a separate branch or controlled development area, then reviewed and integrated into the main codebase.

The workflow was applied as follows:

1. A development task or feature was identified.
2. A feature branch was prepared for the related implementation area.
3. The feature was implemented and tested locally.
4. The completed work was reviewed before integration.
5. Integrated work was merged into the development branch.
6. The final stable version was merged into the main branch for delivery.

This workflow supported code stability, clear change tracking, and controlled integration throughout the project lifecycle.

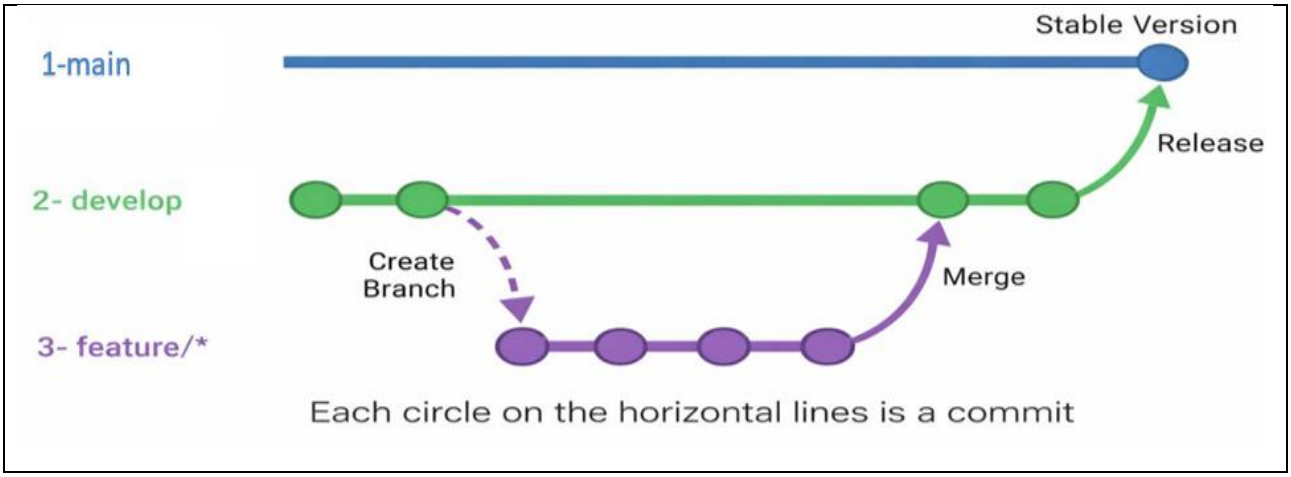

Figure 2.4: SCM Development Workflow

2.8.4 Tags and Release Management

Git tags and GitHub releases were used to mark stable delivery points. The final repository includes release evidence for **V1.0**, which represents a stable project release after integration from the develop branch.

Release management was used to document a validated system state and provide a clear reference point for final submission. Only stable and integrated code should be tagged as an official release.

Table 2.6: Release Management

Release	Description
V1.0	First stable integrated release of the Smart Opinion Survey Platform.
Release Source	Created after merging the develop branch into the main branch.
Purpose	Marks the final delivery version used for academic submission and evaluation.

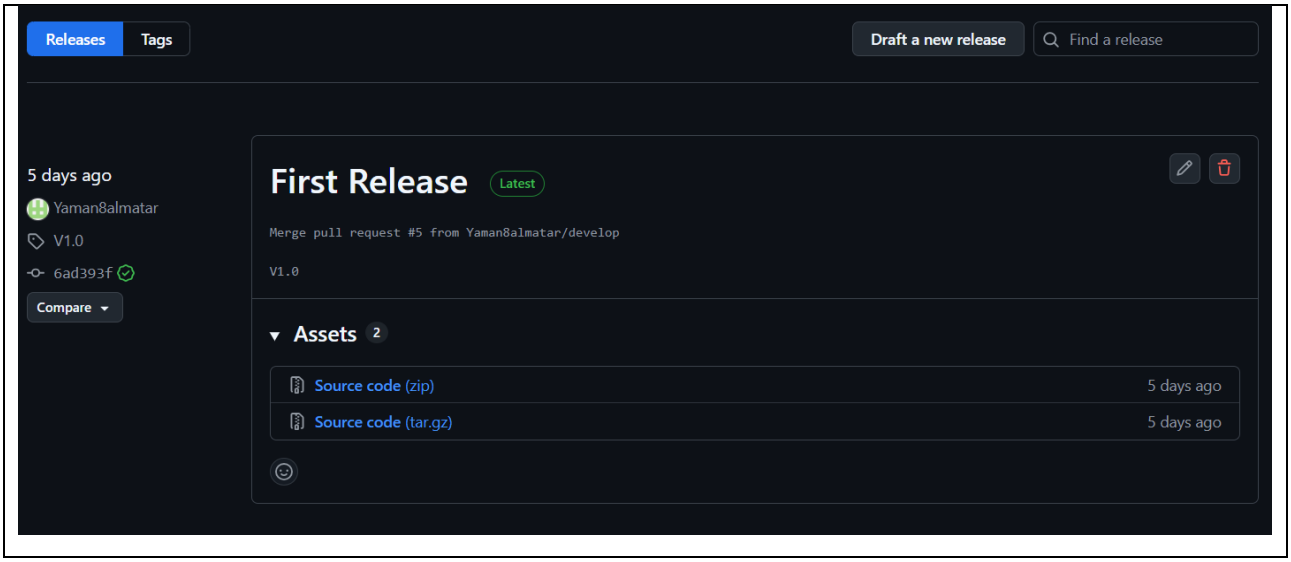

Figure 2.5: GitHub Release Version V1.0

2.8.5 Configuration Items Controlled

The project controlled several configuration items to ensure repeatability and maintainability. These items include source code files, dependency files, database migrations, environment configuration examples, documentation files, and test files.

Sensitive values such as secret keys, database credentials, and Hugging Face API tokens are not stored directly in the repository. Instead, they are configured through environment variables and documented using [.env.example].

This SCM approach supports maintainability, collaboration, traceability, and final delivery control. It also ensures that the submitted system can be reviewed as a stable software product rather than an uncontrolled collection of source files.

Section Two: Theoretical Study and Analysis

Chapter 3: Literature Review and Related Work

3.1 Previous Studies in Smart and Similar Systems

Recent literature shows that online surveys are efficient for collecting responses, but they may suffer from weak sample quality, uncontrolled participation, and limited representativeness. The AAPOR Task Force emphasized that online sample quality should not be evaluated only through response or completion rates, but also through representativeness, inferential reliability, recruitment method, and methodological transparency. Wu et al. also showed that online survey response quality is affected by how clearly the target population is defined, not only by the number of invitations sent.

Other studies focus on questionnaire data quality and sentiment analysis. Jaeger highlighted that online questionnaire quality is affected by questionnaire design, respondent characteristics, distraction, carelessness, and data-quality metrics. Sentiment analysis research shows that open-text feedback can be converted into structured indicators using machine learning, deep learning, and transformer-based models such as BERT.

Based on these studies, the proposed project combines two directions: controlled respondent eligibility and automated open-text analysis. Unlike studies that focus only on online survey quality or only on sentiment classification, this project implements a complete web-based survey platform with demographic targeting, duplicate-response prevention, sentiment analysis, dashboards, and software engineering traceability.

.

Table 3.1: Literature Review Summary

Reference	Focus	Main Contribution	Gap Relevant to This Project
AAPOR Task Force, 2023 [3]	Online sample quality	Defines quality considerations for online samples, including representativeness, recruitment, and reporting transparency.	Does not provide an implemented academic software platform.
Wu et al., 2022 [4]	Online survey response rates	Provides a meta-analysis of response rates in published online survey research.	Focuses on response rates rather than integrated eligibility filtering and AI analysis.
Jaeger, 2022 [5]	Online questionnaire data quality	Identifies factors and metrics affecting the reliability and validity of online questionnaire data.	Does not implement respondent-side demographic eligibility filtering.

Reference	Focus	Main Contribution	Gap Relevant to This Project
Tan et al., 2023 [6]	Sentiment analysis methods	Reviews sentiment analysis approaches, datasets, preprocessing, classifiers, and future research directions.	Focuses on AI methods rather than full survey workflow integration.
Language Representation Model Study, 2023 [7]	Sentiment analysis of customer feedback	Applies machine learning and BERT-based language representation models to airline review sentiment classification.	Domain-specific analysis; not a complete targeted survey platform.
Samir et al., 2023 [8]	Deep learning sentiment analysis	Applies deep learning models to airline customer feedback sentiment classification.	Does not address survey targeting, duplicate prevention, dashboards, or requirements traceability.

This review shows that existing studies address parts of the problem separately. Survey research focuses on data quality and sample reliability, while sentiment analysis research focuses on classifying textual feedback. The Smart Opinion Survey Platform addresses the gap by combining demographic targeting, controlled participation, duplicate prevention, and AI-supported feedback analysis within one engineered software system.

3.2 Modern Techniques and Frameworks in the Field

Modern smart survey systems commonly combine web frameworks, relational databases, responsive user interfaces, dashboard visualization, automated testing, and external AI services. In this project, **Django** was selected as the main web framework because it supports rapid development, clean design, authentication, templates, forms, and database-driven applications. **PostgreSQL** was used as the relational database engine because the system depends on structured entities, relationships, foreign keys, and integrity constraints.

The user interface was implemented using **Django Templates**, **Bootstrap**, and **Bootstrap Icons** to provide server-rendered responsive pages. **Chart.js** was used to visualize survey results and platform metrics because it provides flexible chart types suitable for dashboards. Automated verification was supported through **pytest** and **pytest-django**, which are appropriate for testing service-layer business rules and repository-backed workflows.

For the intelligent component, the system uses the **Hugging Face Inference API** to perform sentiment analysis on open-text answers. This approach is suitable for the project because it allows the system to use an existing text-classification model instead of training a machine learning model from scratch. Hugging Face documentation defines text classification as assigning labels to text, including use cases such as sentiment analysis.

3.3 Comparative Analysis of Existing Systems

Table 3.2: Comparative Analysis of Existing Systems

Criterion	Google Forms	SurveyMonkey Audience	Qualtrics Panels	Proposed Platform
Main Purpose	General form and survey creation.	Paid targeted survey respondents.	Commercial research panels.	Academic smart survey platform.
Audience Targeting	Mainly through sharing and access settings.	Uses paid targeting options and panelists.	Provides targeted panel services.	Uses respondent demographic profiles.
AI Text Analysis	Not a core standard workflow.	Available depending on advanced features.	Available in advanced ecosystems.	Integrated sentiment analysis.
Duplicate Control	Configurable in limited cases.	Controlled by platform workflow.	Controlled by panel workflow.	Service validation and database constraint.
Transparency	Closed commercial system.	Closed commercial system.	Closed commercial system.	Documented architecture, tests, and traceability.

The comparison shows that existing systems provide strong survey or panel services, but the proposed platform combines demographic targeting, duplicate prevention, sentiment analysis, and academic software engineering traceability in one implemented system.

3.4 Research Gap and Proposed Innovation

The identified gap is the absence of a lightweight academic system that combines respondent-side demographic profiles, integrated survey eligibility matching, duplicate-response control, AI-assisted sentiment analysis, dashboards, and software engineering traceability in one implemented platform.

The proposed innovation is not a new machine learning algorithm. Instead, the innovation is the **architectural and functional integration** of demographic targeting, controlled participation, sentiment analysis, and decision-support dashboards within a complete software engineering project. This makes the system suitable for academic evaluation because the project can be assessed through requirements, design, implementation, testing, and traceability evidence.

3.5 Theoretical Framework

The project is based on the following theoretical and engineering concepts:

- **Two-sided platform:** value is produced through interaction between service providers who create surveys and respondents who answer them.
- **Demographic targeting:** respondent eligibility is derived from profile attributes such as age, gender, and region.

- **Sentiment analysis:** open-text answers are classified into positive, neutral, or negative sentiment labels.
- **Layered architecture:** presentation, services, repositories, domain models, and infrastructure are separated to improve maintainability.
- **Requirements traceability:** requirements are mapped to use cases, implementation areas, and test cases to support verification.

.

3.6 Modern and Future Trends

Future survey platforms are moving toward stronger respondent segmentation, automated insight extraction, quality metrics, dashboard-based reporting, and AI-supported analysis. In this direction, the Smart Opinion Survey Platform can be extended with asynchronous AI processing, stronger respondent quality controls, topic modeling, richer analytics, audit logs, privacy controls, and CI/CD automation.

These improvements are treated as future work, not as current implemented features. This distinction is important because the final report must not claim features that are not implemented in the current system version.

Chapter 4: Requirements Modeling and Analysis

4.1 Feasibility Study

The feasibility study evaluates whether the Smart Opinion Survey Platform can be built and operated within the available technical, operational, economic, and schedule constraints. The project is feasible because it uses available open-source frameworks, a clear role-based workflow, and an external AI service instead of training a model from scratch. Django is suitable for rapid and clean web application development, PostgreSQL provides a stable relational database environment, pytest-django supports Django testing with pytest, and Hugging Face supports text classification tasks such as sentiment analysis.

Table 4.1: Feasibility Study

Feasibility Type	Assessment
Technical Feasibility	Feasible because Django 4.2.11, PostgreSQL 15, Bootstrap, Chart.js, pytest, requests, and Hugging Face Inference API are available and appropriate for the project scope.
Operational Feasibility	Feasible because the system actors are clear: service provider, respondent, and administrator.
Economic Feasibility	Feasible because the project uses open-source frameworks and an external AI API instead of training a model from scratch.
Schedule Feasibility	Feasible because the work was organized into Sprint 0 and four implementation sprints.

4.2 Requirements Collection

Requirements were collected from the project idea, seminar discussions, approved use cases, the official university report guide, and inspection of the implemented Django repository. The collection process focused on identifying the needs of the main actors: service providers, respondents, administrators, academic supervisors, and future maintainers.

The collected requirements were then refined into functional requirements, non-functional requirements, use cases, user stories, and test cases. This ensured that the report did not describe the system only at a conceptual level, but linked each requirement to analysis, design, implementation, and testing evidence.

4.3 Stakeholders and Needs Analysis

Stakeholder analysis identifies the main parties affected by the system and clarifies their needs. The Smart Opinion Survey Platform has three primary operational stakeholders: service providers, respondents, and administrators. It also has academic and maintenance stakeholders, because the project must be evaluated, documented, and maintained as a software engineering system.

Table 4.2: Stakeholder Analysis

Stakeholder	Need	System Response
Service Provider	Create targeted surveys and obtain reliable feedback analytics.	Survey creation, question management, targeting criteria, publication, closing, and analysis dashboard.
Respondent	Participate only in surveys relevant to their demographic profile.	Registration, demographic profile management, eligible survey listing, response submission, and answered survey history.
System Administrator	Manage user accounts and monitor platform activity.	User management, account status actions, and system reports.
Academic Supervisors	Review and evaluate the project quality and academic completeness.	Review of methodology, requirements, architecture, documentation, testing evidence, and final deliverables.

4.4 Functional Requirements

Table 4.3: Functional Requirements

ID	Functional Requirement	Actor
FR-01	The system shall allow a service provider to create an account using name, email, and password.	Service Provider
FR-02	The system shall allow a respondent to create an account with demographic data.	Respondent
FR-03	The system shall allow users to log in using email and password.	Service Provider, Respondent, System Administrator
FR-04	The system shall allow authenticated users to log out.	Service Provider, Respondent, Administrator

ID	Functional Requirement	Actor
FR-05	The system shall allow service providers to create surveys with title and description.	Service Provider
FR-06	The system shall allow service providers to update their own draft survey details.	Service Provider
FR-07	The system shall allow service providers to delete their own draft or closed surveys.	Service Provider
FR-08	The system shall support multiple-choice, rating-scale, and open-text question types.	Service Provider, Respondent
FR-09	The system shall allow service providers to add questions to their own draft surveys.	Service Provider
FR-10	The system shall support deleting questions on owned draft surveys.	Service Provider
FR-11	The system shall allow service providers to define demographic targeting criteria for their own draft surveys.	Service Provider
FR-12	The system shall allow service providers to publish surveys only when required publication conditions are satisfied.	Service Provider
FR-13	The system shall allow service providers to close published surveys.	Service Provider
FR-14	The system shall allow service providers to view analytics for their own surveys.	Service Provider
FR-15	The system shall allow respondents to manage their demographic profile.	Respondent
FR-16	The system shall list only eligible published surveys for a respondent.	Respondent
FR-17	The system shall allow respondents to submit responses to eligible published surveys.	Respondent
FR-18	The system shall prevent a respondent from submitting more than one response to the same survey.	Respondent
FR-19	The system shall allow respondents to view surveys they have already answered.	Respondent
FR-20	The system shall allow administrators to activate, suspend, and soft-delete user accounts.	System Administrator
FR-21	The system shall allow administrators to generate system reports for platform activity.	System Administrator
FR-22	The system shall run sentiment analysis for open-text answers and store the result state.	System

4.5 Non-Functional Requirements

Table 4.4: Non-Functional Requirements

NFR ID	Category	Requirement	Measurable Criterion / Verification
NFR-01	Security	The system shall protect account passwords using Django password hashing.	Passwords are stored hashed, not plaintext.
NFR-02	Security	The system shall restrict protected functions according to user roles.	Service Provider, Respondent, and Administrator workflows are separated by role checks.
NFR-03	Security	The system shall load real runtime secrets and API tokens from environment variables.	Database settings and AI API token are loaded from environment variables; `.env` is not committed.
NFR-04	Usability	The system shall provide role-specific navigation and dashboard pages.	Each role has relevant navigation links and dashboard pages.
NFR-05	Usability	The system shall provide clear forms and feedback messages for main workflows.	Forms validate user input and display success or error messages.
NFR-06	Performance	The system should return normal local page requests within 2 seconds during academic testing.	Target to be verified manually or with browser timing before final submission.
NFR-07	Reliability	The system shall keep a submitted response saved if sentiment analysis fails.	Failed external analysis is recorded as `FAILED` without deleting the submitted response.
NFR-08	Maintainability	The system shall separate UI, services, repositories, models, and external integrations.	The implementation follows Views → Services → Repositories → Models/ORM → Database.
NFR-09	Data Integrity	The system shall prevent duplicate responses for the same survey and respondent.	One response per survey/respondent is enforced by validation and a database constraint.
NFR-10	Privacy	The system shall store respondent demographic data on the server side for profile and survey eligibility workflows.	Demographic data is used for profile management and eligibility matching only.

4.6 Requirement Prioritization and Management

Table 4.5: MoSCoW Requirement Prioritization

Priority	Requirements
Must	Authentication, role-based access, survey CRUD, question management, targeting criteria, survey publication/closing, eligible survey filtering, response submission, duplicate prevention, sentiment analysis, and provider analysis dashboard.
Should	Admin account management, system reports, service-layer tests, and clear dashboard-based result presentation.
Could	CI/CD workflow, accessibility audit, performance testing, audit logs, richer analytics, and asynchronous AI processing.
Won't	Native mobile application, payment/reward system, training a machine learning model from scratch, public REST API, and large-scale production deployment.

4.7 UML Requirement Diagrams

4.7.1 Main Use Case Diagram

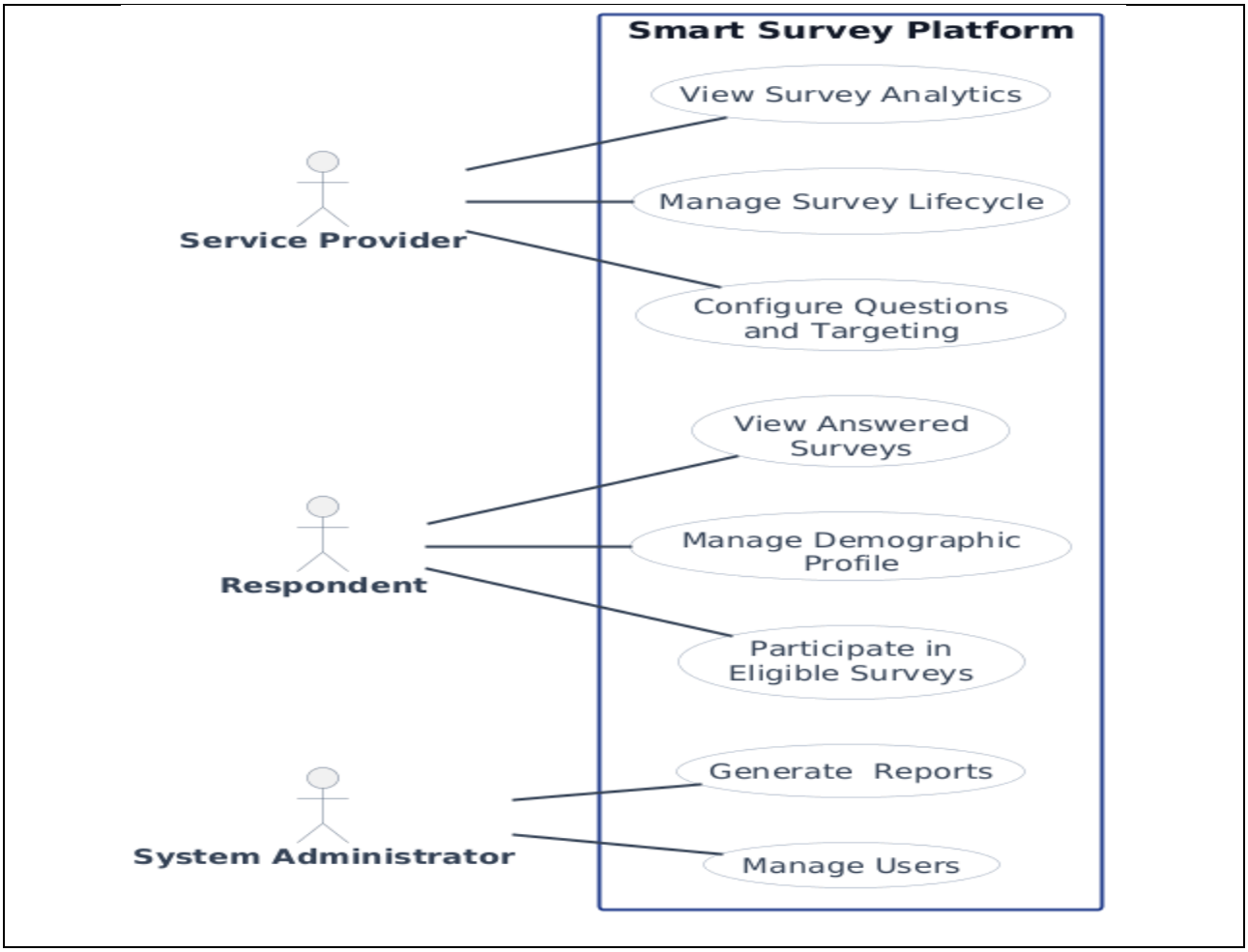

Figure 4.1: Main Use Case Diagram

4.7.2 Use Case Descriptions

Table 4.6: Main Use Case Specifications

Field	Description
Use Case ID and Name	**UC-M01: Manage Survey Lifecycle**
Primary Actor	Service Provider
Brief Description	This use case allows a service provider to create, update, delete, publish, and close surveys according to their lifecycle status.
Main Flow	1. The service provider opens the survey management page. 2. The system displays the provider's surveys and available actions. 3. The provider creates a new survey with title and description. 4. The system saves the survey as a draft. 5. The provider updates or deletes the survey while it is still in draft status when needed. 6. The provider publishes the survey after completing the required questions and targeting criteria. 7. The provider closes the published survey when response collection is complete.
Postconditions	1. The survey status is correctly maintained as draft, published, or closed. 2. Published surveys become available to eligible respondents. 3. Closed surveys no longer accept responses.
Alternative Flow	The provider may keep the survey in draft status and complete questions or targeting criteria later.
Exceptions	1. The system rejects update or delete actions if the survey is not in draft status. 2. The system rejects publication if the survey has no questions or no targeting criteria. 3. The system rejects closing if the survey is not published.

Field	Description
Use Case ID and Name	**UC-M02: Configure Questions and Targeting**
Primary Actor	Service Provider
Brief Description	This use case allows a service provider to add survey questions and define demographic targeting criteria for an owned draft survey.

Field	Description
Main Flow	1. The service provider opens the question or targeting management page for a draft survey. 2. The system validates provider ownership and draft status. 3. The provider adds questions using supported question types: multiple-choice, rating-scale, or open-text. 4. The system validates question text, question type, required flag, order, and options when needed. 5. The system saves the question and related options. 6. The provider defines targeting criteria using gender, age range, and region. 7. The system validates and stores the targeting criteria.
Postconditions	1. The survey contains valid questions. 2. The survey has targeting criteria used for respondent eligibility filtering. 3. The survey becomes closer to publication readiness.
Alternative Flow	The provider may leave optional targeting fields empty when no restriction is required for that field.
Exceptions	1. The system rejects question changes if the survey is not in draft status. 2. The system rejects invalid multiple-choice options. 3. The system rejects invalid targeting criteria such as an invalid age range.

Field	Description
Use Case ID and Name	**UC-M03: View Survey Analytics**
Primary Actor	Service Provider
Brief Description	This use case allows a service provider to view response counts, sentiment results, and question summaries for an owned survey.
Main Flow	1. The service provider opens the analytics page for a selected survey. 2. The system validates that the survey belongs to the provider. 3. The system loads submitted responses and answers. 4. The system loads sentiment analysis results for open-text answers. 5. The system calculates response counts, sentiment counts, rates, and question summaries. 6. The system displays the analytics dashboard.
Postconditions	1. The provider views survey analytics. 2. The survey data remains unchanged.
Alternative Flow	If no responses exist, the system displays empty or zero-value analytics.
Exceptions	The system rejects access if the survey does not exist or does not belong to the provider.

Field	Description
Use Case ID and Name	**UC-M04: Manage Demographic Profile**
Primary Actor	Respondent
Brief Description	This use case allows a respondent to view or update demographic profile data used for survey eligibility.
Main Flow	1. The respondent opens the demographic profile page. 2. The system displays the current profile data if available. 3. The respondent enters or updates age, gender, region, and interests. 4. The respondent submits the form. 5. The system validates the profile data. 6. The system saves the updated demographic profile.
Postconditions	1. The respondent profile is stored or updated. 2. Future eligibility checks use the latest demographic data.
Alternative Flow	The respondent may view the profile without submitting changes.
Exceptions	The system rejects invalid profile data and displays validation messages.

Field	Description
Use Case ID and Name	**UC-M05: Participate in Eligible Surveys**
Primary Actor	Respondent
Brief Description	This use case allows a respondent to view eligible surveys and submit a response to an eligible published survey.
Main Flow	1. The respondent opens the eligible surveys page. 2. The system loads the respondent demographic profile. 3. The system retrieves published surveys. 4. The system applies targeting criteria and removes surveys already answered by the respondent. 5. The respondent selects an eligible survey. 6. The system displays ordered questions and options. 7. The respondent submits answers. 8. The system validates eligibility, duplicate response status, required answers, and answer formats. 9. The system saves the response and answers. 10. The system runs sentiment analysis for open-text answers when applicable.

Field	Description
Postconditions	1. A response and answer set are stored. 2. The respondent cannot submit another response to the same survey. 3. Open-text sentiment analysis results are completed or marked failed.
Alternative Flow	If no eligible surveys exist, the system displays an empty list message.
Exceptions	1. The system rejects submission if the respondent is not eligible. 2. The system rejects duplicate responses. 3. The system rejects missing required answers or invalid answer formats. 4. If sentiment analysis fails, the response remains saved, and the analysis result is marked failed.

Field	Description
Use Case ID and Name	**UC-M06: View Answered Surveys**
Primary Actor	Respondent
Brief Description	This use case allows a respondent to view surveys previously answered by the respondent.
Main Flow	1. The respondent opens the answered surveys page. 2. The system validates that the user is a respondent. 3. The system loads responses submitted by the respondent. 4. The system builds a history list with survey title, description, and submission date. 5. The system displays the answered survey history.
Postconditions	1. The respondent sees previously answered surveys. 2. No response data is changed.
Alternative Flow	If no answered surveys exist, the system displays an empty history state.
Exceptions	The system rejects access if the user is not a respondent.

Field	Description
Use Case ID and Name	**UC-M07: Manage Users and Reports**
Primary Actor	System Administrator
Brief Description	This use case allows a system administrator to manage user account status and view platform activity reports.
Main Flow	1. The administrator opens the administration area. 2. The system validates administrator access. 3. The system displays user accounts with available actions. 4. The administrator activates, suspends, or soft-deletes a selected user account. 5. The system validates the action and prevents self-destructive actions. 6. The system updates the selected account status. 7. The administrator opens the system reports page. 8. The system validates the report date range. 9. The system aggregates user, survey, response, growth, and most-active-survey metrics. 10. The system displays report cards and charts.
Postconditions	1. Valid account status actions are applied. 2. The administrator views system report metrics. 3. Platform data remains consistent.
Alternative Flow	If no date range is entered, the system uses the default reporting range.
Exceptions	1. The system rejects access for non-administrators. 2. The system rejects suspending or deleting the administrator’s own account. 3. The system rejects invalid report date ranges.

4.7.3 Activity Diagrams

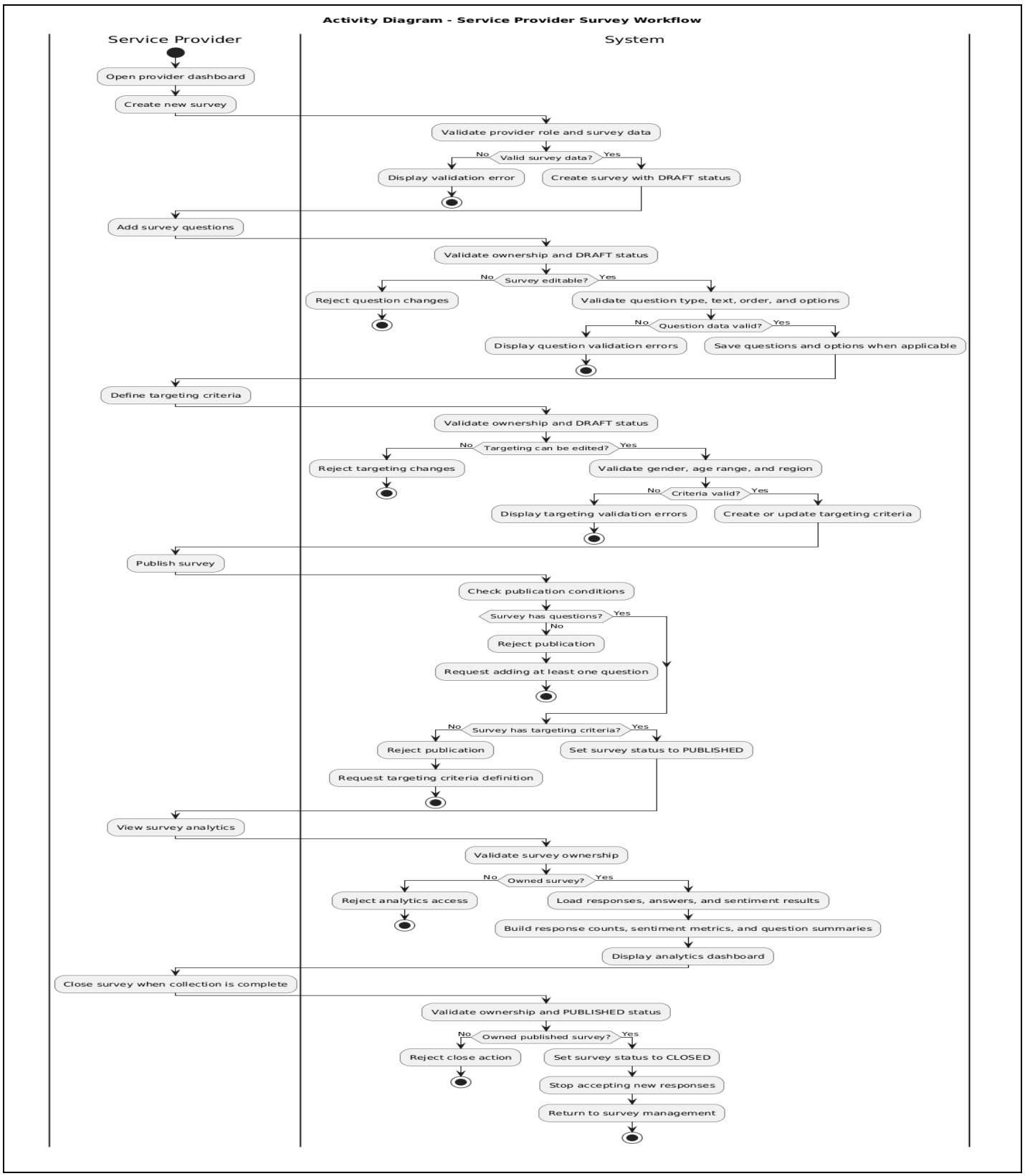

Figure 4.2: Activity Diagram - Service Provider Survey Workflow

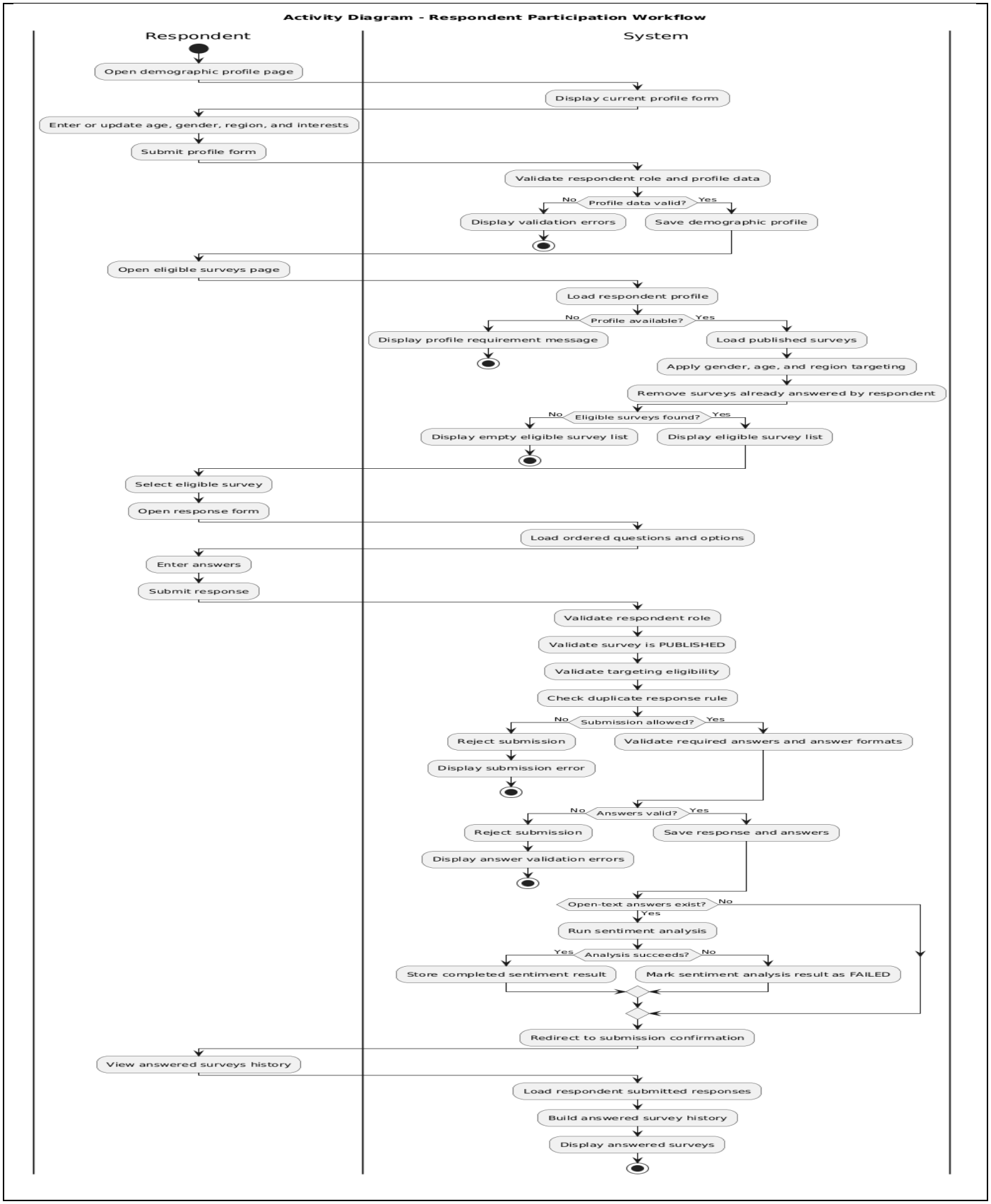

Figure 4.3: Activity Diagram - Respondent Participation Workflow

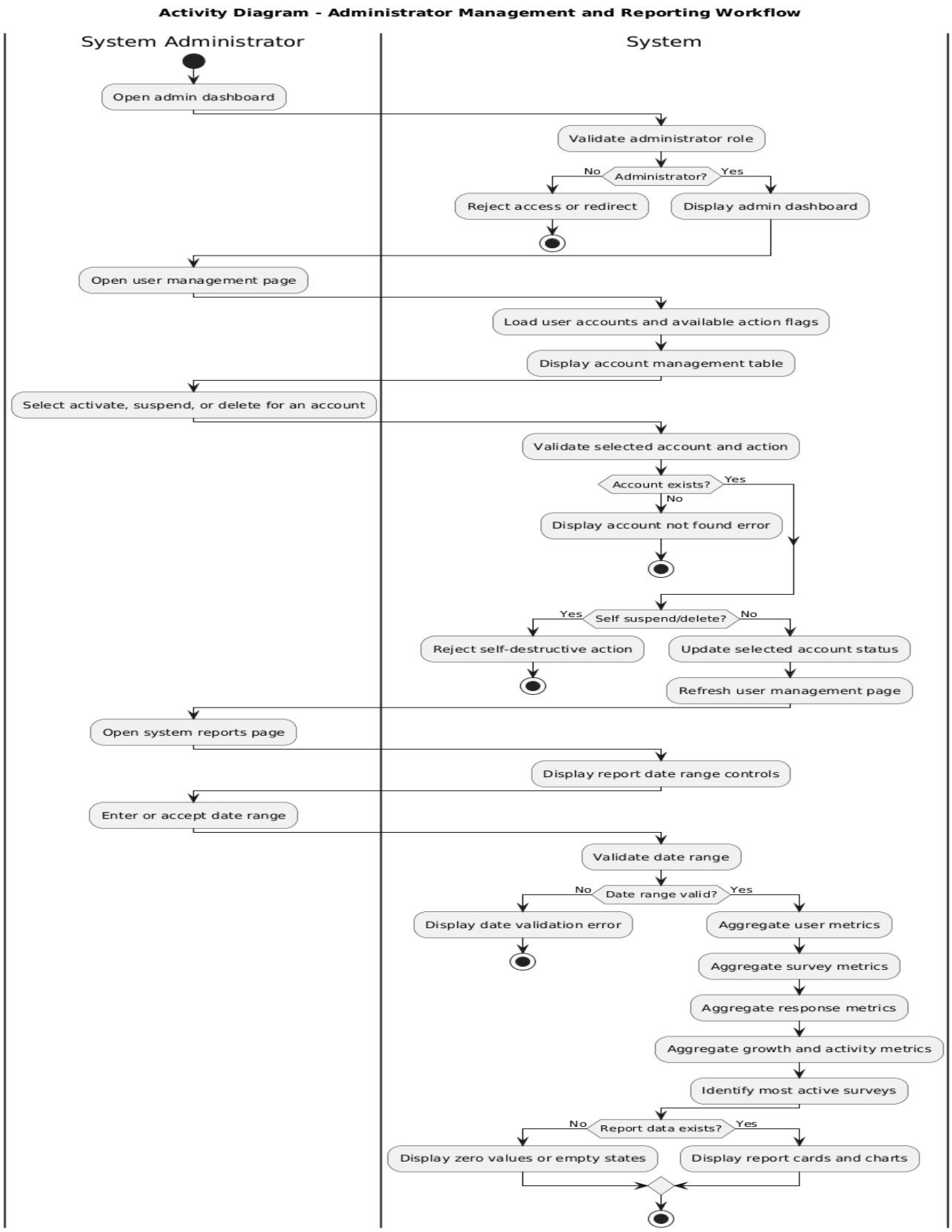

Figure 4.4: Activity Diagram - Administrator Management and Reporting Workflow

4.7.4 Sequence Diagrams

The sequence diagrams focus on the most critical system workflows rather than listing every use case. The selected diagrams represent survey publication, respondent participation with sentiment analysis, provider analytics, and administrator reporting.

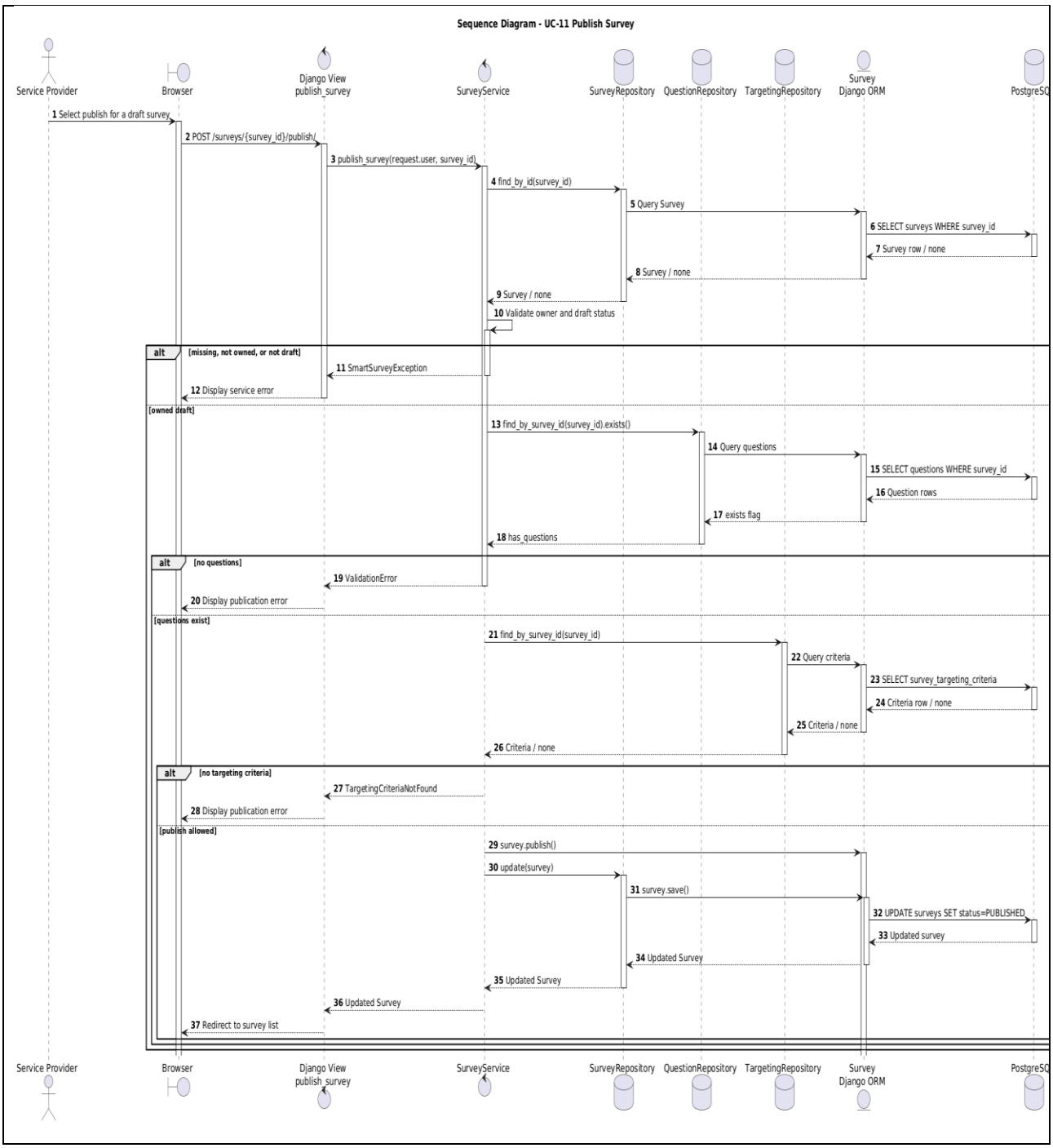

Figure 4.5: Sequence Diagram - Publish Survey

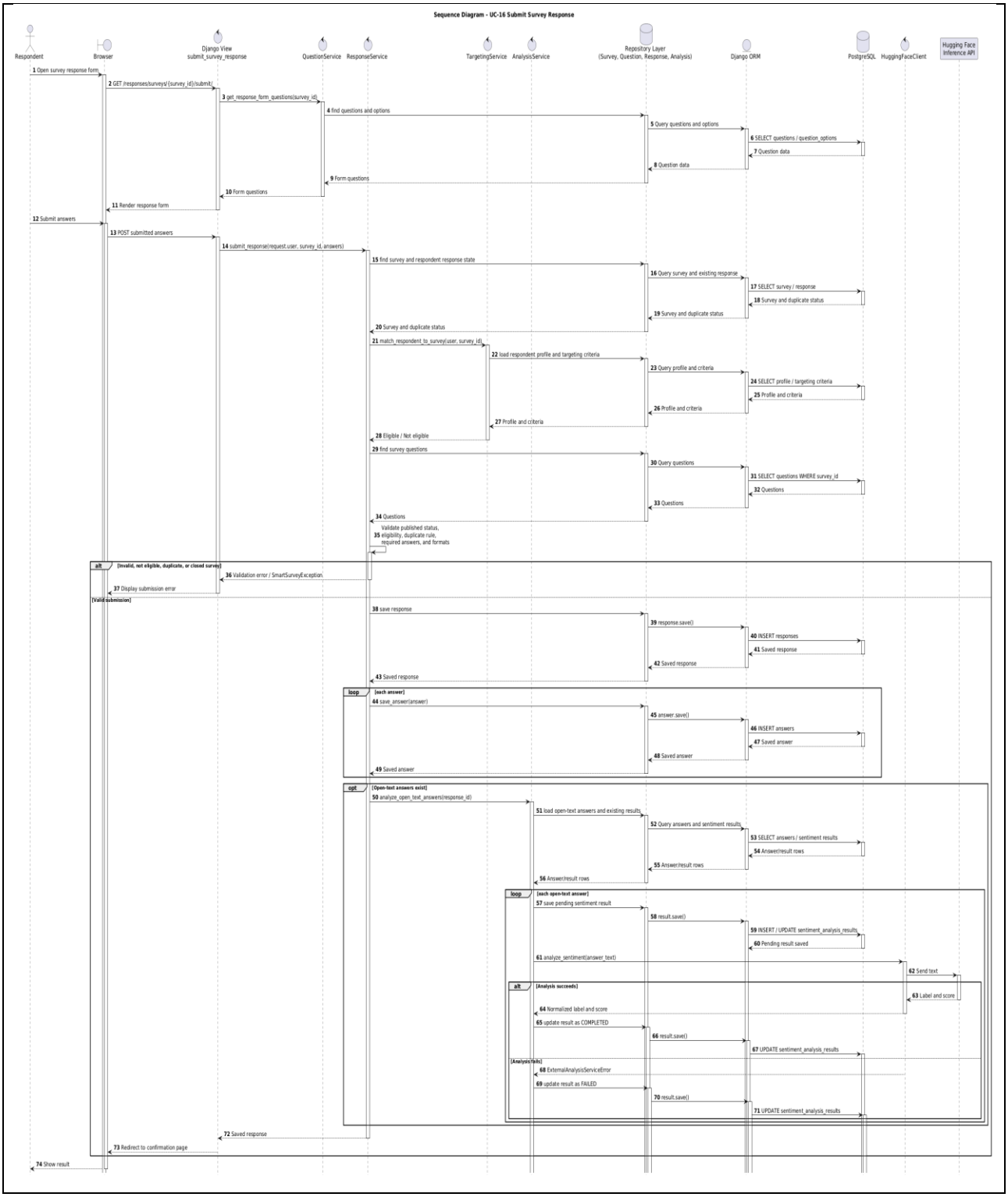

Figure 4.6: Sequence Diagram - Submit Survey Response

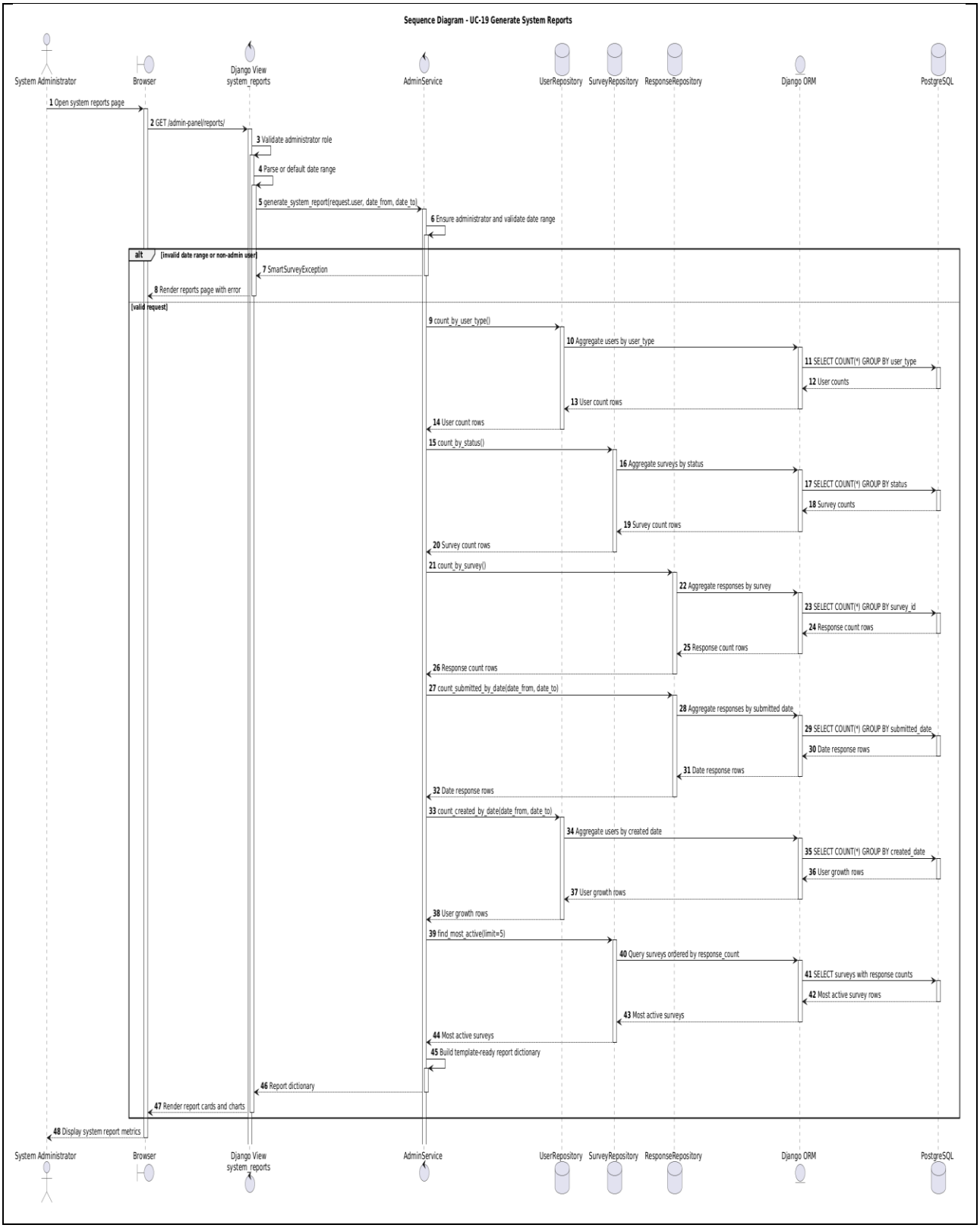

Figure 4.7: Sequence Diagram - Generate System Reports

4.7.5 Test Case Descriptions

Test Case	Scenario	Input / Condition	Expected Result	Related Use Case
TC-01	Register service provider	Valid provider name, email, and password	Service provider account is created successfully	UC-01
TC-02	Register respondent	Valid respondent account and demographic data	Respondent account and demographic profile are created	UC-02
TC-03	Reject duplicate email	Registration request uses an existing email	Validation error is returned, and account is not created	UC-01, UC-02
TC-04	Reject inactive account login	Suspended or deleted user attempts login	Login is rejected and access is blocked	UC-03
TC-05	Add supported question type	Provider adds multiple-choice, rating-scale, or open-text question to draft survey	Question is saved according to its type and validation rules	UC-08
TC-06	Publish incomplete survey	Draft survey has no questions or no targeting criteria	Publication is rejected with validation error	UC-11
TC-07	Filter eligible surveys	Respondent profile and survey targeting criteria exist	Only matching, unanswered, published surveys are displayed	UC-15
TC-08	Reject duplicate response	Respondent already answered the selected survey	Duplicate response error is returned	UC-16
TC-09	Analyze open-text answer	Respondent submits an open-text answer	Sentiment result is stored as completed or failed safely	UC-13, UC-16
TC-10	Admin suspends account	Administrator suspends an active user account	Account status becomes `SUSPENDED`	UC-18

4.8 Modeling According to the Agile Methodology

The project followed an Agile Iterative and Incremental Development methodology. Therefore, modeling focused on product backlog items, user stories, sprint-based feature delivery, and continuous refinement of requirements across iterations. A complete traditional SRS was not used as the primary management model; however, the functional and non-functional requirements were documented in a structured form to support academic traceability.

4.8.1 Product Backlog and User Stories

Table 4.7: Product Backlog Summary

Backlog ID	Backlog Item	Related Use Case	Priority	Sprint
PB-01	Register service providers and respondents.	UC-01, UC-02	Must	Sprint 1
PB-02	Authenticate users and redirect them by role.	UC-03, UC-04	Must	Sprint 1
PB-03	Create and manage surveys.	UC-05, UC-06, UC-07	Must	Sprint 2
PB-04	Add survey questions with supported question types.	UC-08	Must	Sprint 2
PB-05	Define demographic targeting criteria.	UC-10	Must	Sprint 2
PB-06	Publish and close surveys.	UC-11, UC-12	Must	Sprint 2
PB-07	Display eligible surveys to respondents.	UC-15	Must	Sprint 3
PB-08	Submit survey responses and prevent duplicates.	UC-16	Must	Sprint 3
PB-09	Analyze open-text answers using sentiment analysis.	UC-16, UC-13	Must	Sprint 3
PB-10	Display provider analytics dashboard.	UC-13	Must	Sprint 4
PB-11	Manage user accounts by administrator.	UC-18	Should	Sprint 4
PB-12	Generate system reports.	UC-19	Should	Sprint 4

Table 4.8: User Stories

Story ID	User Story	Acceptance Criteria
US-01	As a service provider, I want to create surveys so that I can collect feedback from respondents.	The provider can create a draft survey with title and description.
US-02	As a service provider, I want to add questions and targeting criteria so that the survey reaches the correct audience.	Questions and targeting criteria are saved only for owned draft surveys.
US-03	As a service provider, I want to publish a complete survey so that eligible respondents can answer it.	Publication is allowed only when questions and targeting criteria exist.
US-04	As a respondent, I want to view only eligible surveys so that I participate in relevant surveys.	The displayed surveys match the respondent profile and exclude answered surveys.
US-05	As a respondent, I want to submit a survey response so that my feedback is recorded.	The response is saved once, and duplicate submissions are rejected.
US-06	As a service provider, I want to view analytics so that I can understand survey results.	The dashboard shows response metrics and sentiment analysis results.
US-07	As an administrator, I want to manage user accounts so that platform access is controlled.	The administrator can activate, suspend, or soft-delete user accounts.
US-08	As an administrator, I want to generate system reports so that platform activity can be monitored.	Reports show user, survey, and response metrics.

4.8.2 SRS Support for Academic Traceability

Although the project was not managed using a traditional waterfall methodology, SRS-style documentation was used to structure the functional requirements, non-functional requirements, actors, use cases, and traceability matrix. This helped maintain academic clarity and ensured that implementation and testing could be mapped back to documented requirements.

Table 4.9: Iterative Feature List

Iteration / Sprint	Implemented Feature Group	Main Output
Sprint 0	Discovery, requirement analysis, backlog setup, and architecture baseline.	MVP scope, initial requirements, use cases, and architecture plan.
Sprint 1	Authentication, roles, and respondent profile foundation.	Registration, login/logout, role handling, and demographic profile structure.
Sprint 2	Survey management, question management, targeting, and publication workflow.	Survey CRUD, question creation, targeting criteria, publish/close logic.
Sprint 3	Respondent participation and intelligent analysis.	Eligible survey filtering, response submission, duplicate prevention, and sentiment analysis.
Sprint 4	Dashboards, administration, reporting, testing, and documentation.	Analytics dashboard, admin user management, system reports, tests, and final report evidence.

4.9 Requirements Analysis and Documentation

Requirements traceability was used to ensure that each functional requirement is connected to the corresponding design component and testing scenario. This supports verification and prevents implementation features from becoming disconnected from the documented requirements.

Table 4.10: Requirements Traceability Matrix

Req. ID	Requirement Summary	Related Use Case	Design / Implementation Component	Related Test Case
FR-01	Register service provider account.	UC-01	AuthenticationService, UserRepository, User model, registration view/form	TC-01
FR-02	Register respondent with demographic data.	UC-02	AuthenticationService, ProfileRepository, User model, RespondentProfile model	TC-02
FR-03	Login using email and password.	UC-03	AuthenticationService, UserRepository, Django authentication/session	TC-03
FR-04	Logout authenticated users.	UC-04	Django logout view and session framework	TC-04
FR-05	Create surveys with title and description.	UC-05	SurveyService, SurveyRepository, Survey model, survey form/view	TC-05
FR-06	Update own draft survey details.	UC-06	SurveyService, SurveyRepository, Survey model	TC-05
FR-07	Delete own draft or closed surveys.	UC-07	urvey.can_delete, SurveyService.delete_survey, survey templates	TC-05
FR-08	Support multiple-choice, rating-scale, and open-text question types.	UC-08, UC-16	Question model, Answer model, QuestionService, ResponseService	TC-06
FR-09	Add questions to owned draft surveys.	UC-08	QuestionService, QuestionRepository, QuestionOption model, question form/view	TC-06
FR-10	Update and delete questions on owned draft surveys.	UC-09	QuestionService, QuestionRepository	TC-07
FR-11	Define demographic targeting criteria.	UC-10	TargetingService, TargetingRepository, TargetingCriteria model	TC-08
FR-12	Publish surveys only when publication conditions are satisfied.	UC-11	SurveyService, QuestionRepository, TargetingRepository, Survey model	TC-09
FR-13	Close published surveys.	UC-12	SurveyService, SurveyRepository, Survey model	TC-09

Req. ID	Requirement Summary	Related Use Case	Design / Implementation Component	Related Test Case
FR-14	View analytics for owned surveys.	UC-13	AnalysisService, ResponseRepository, AnalysisRepository, analytics dashboard	TC-10
FR-15	Manage respondent demographic profile.	UC-14	ProfileService, ProfileRepository, RespondentProfile model, profile form/view	TC-11
FR-16	List only eligible published surveys for respondent.	UC-15	SurveyService, TargetingRepository, ResponseRepository, ProfileRepository	TC-12
FR-17	Submit responses to eligible published surveys.	UC-16	ResponseService, QuestionService, Response model, Answer model	TC-13
FR-18	Prevent duplicate response for same survey/respondent.	UC-16	ResponseService, ResponseRepository, Response unique constraint	TC-13
FR-19	View already answered surveys.	UC-17	ResponseService, ResponseRepository, answered surveys view/template	TC-14
FR-20	Activate, suspend, and soft-delete user accounts.	UC-18	AdminService, UserRepository, User model, admin user management view	TC-15
FR-21	Generate system reports for platform activity.	UC-19	AdminService, UserRepository, SurveyRepository, ResponseRepository, reports dashboard	TC-16
FR-22	Run sentiment analysis for open-text answers and store result state.	UC-16, UC-13	AnalysisService, HuggingFaceClient, AnalysisRepository, SentimentAnalysisResult model	TC-17

Table 4.11: Initial Test Cases

Test Case	Scenario	Input / Condition	Expected Result	Related Req. ID
TC-01	Register service provider	Valid provider name, email, and password	Service provider account is created	FR-01
TC-02	Register respondent	Valid account and demographic data	Respondent account and profile are created	FR-02
TC-03	Login validation	Valid credentials, invalid credentials, suspended/deleted account	Valid active user logs in; invalid or inactive users are rejected	FR-03

Test Case	Scenario	Input / Condition	Expected Result	Related Req. ID
TC-04	Logout user	Authenticated user requests logout	Session is ended and user is redirected	FR-04
TC-05	Survey draft CRUD	Provider creates, updates, or deletes own draft survey	Draft survey is created, updated, or deleted according to status rules	FR-05, FR-06, FR-07
TC-06	Add supported question types	Provider adds multiple-choice, rating-scale, or open-text question	Question is saved according to type and validation rules	FR-08, FR-09
TC-07	Update/delete question service rule	Provider updates or deletes a question on an owned draft survey	Service accepts valid draft operations and rejects invalid ones	FR-10
TC-08	Define targeting criteria	Provider submits gender, age range, or region criteria	Targeting criteria are created or updated	FR-11
TC-09	Publish and close survey	Draft survey with questions and targeting; published survey to close	Complete draft is published; published survey is closed	FR-12, FR-13
TC-10	View survey analytics	Provider opens analytics for owned survey	Response counts, sentiment metrics, and summaries are displayed	FR-14
TC-11	Manage demographic profile	Respondent submits valid profile data	Profile is saved and used for eligibility	FR-15
TC-12	Filter eligible surveys	Respondent profile and targeting criteria exist	Only matching unanswered published surveys are displayed	FR-16
TC-13	Submit response and prevent duplicate	Respondent submits answers, then tries again	First response is saved; duplicate submission is rejected	FR-17, FR-18
TC-14	View answered surveys	Respondent has submitted responses	Answered survey history is displayed	FR-19
TC-15	Manage user accounts	Administrator activates, suspends, or soft-deletes account	Account status is updated, and self-destructive actions are rejected	FR-20
TC-16	Generate system reports	Administrator selects or accepts date range	Platform report metrics are generated and displayed	FR-21
TC-17	Analyze open-text answer	Response contains open-text answer	Sentiment result is stored as completed or failed safely	FR-22

Section Three: Design and Architecture

Chapter 5: System Architectural Design

5.1 High-Level Design

The Smart Survey Platform is implemented as a Django web application using Django MVT combined with an explicit layered architecture. Django MVT is used to structure web presentation through views, templates, and models, while the layered architecture separates presentation, business logic, data access, domain models, and external AI integration.

The presentation layer contains Django views, forms, templates, Bootstrap, and Chart.js. The service layer contains business rules, validation, authorization checks, and workflow orchestration. The repository layer isolates database operations through Django ORM. The domain/data layer contains the persistent models, relationships, constraints, and helper methods. The infrastructure layer isolates communication with the Hugging Face Inference API.

5.1.1 Architectural Block Diagram

The system block diagram illustrates the main components of the Smart Survey Platform and how they interact. Users access the system through a web browser. The browser communicates with Django views, which render templates, process forms, and call service classes. Service classes apply business rules and call repositories. Repositories access Django ORM models, which are mapped to the PostgreSQL database. The AI component is isolated through HuggingFaceClient, which communicates with the Hugging Face Inference API.

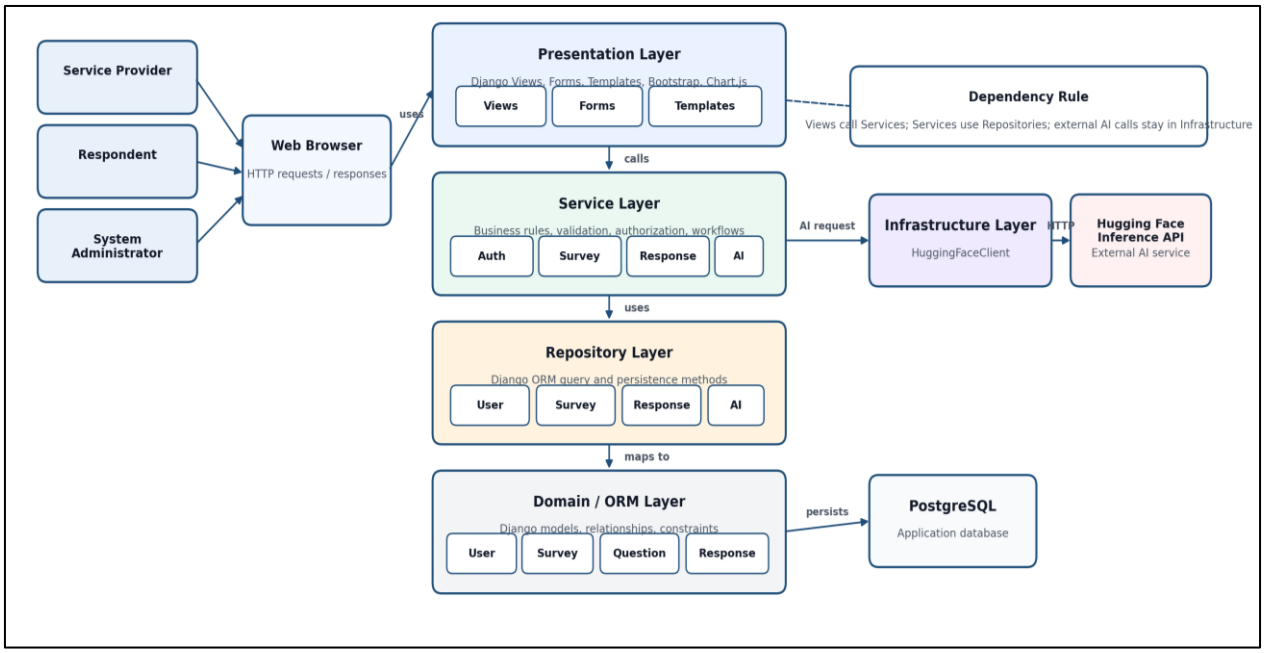

Figure 5.1: System Block Diagram

5.1.2 Architectural Pattern Used

The architectural pattern used in this project is **Layered Architecture** implemented inside a Django MVT web application. Django MVT provides the web framework structure, while the layered architecture enforces separation of concerns between presentation, business logic, data access, domain models, and infrastructure.

The main dependency direction is:

Views → Services → Repositories → Django ORM Models → PostgreSQL

The intelligent analysis dependency direction is:

ResponseService → AnalysisService → HuggingFaceClient → Hugging Face Inference API

The system does not use a microservices architecture. It is implemented as a modular monolithic Django application with strict internal layers.

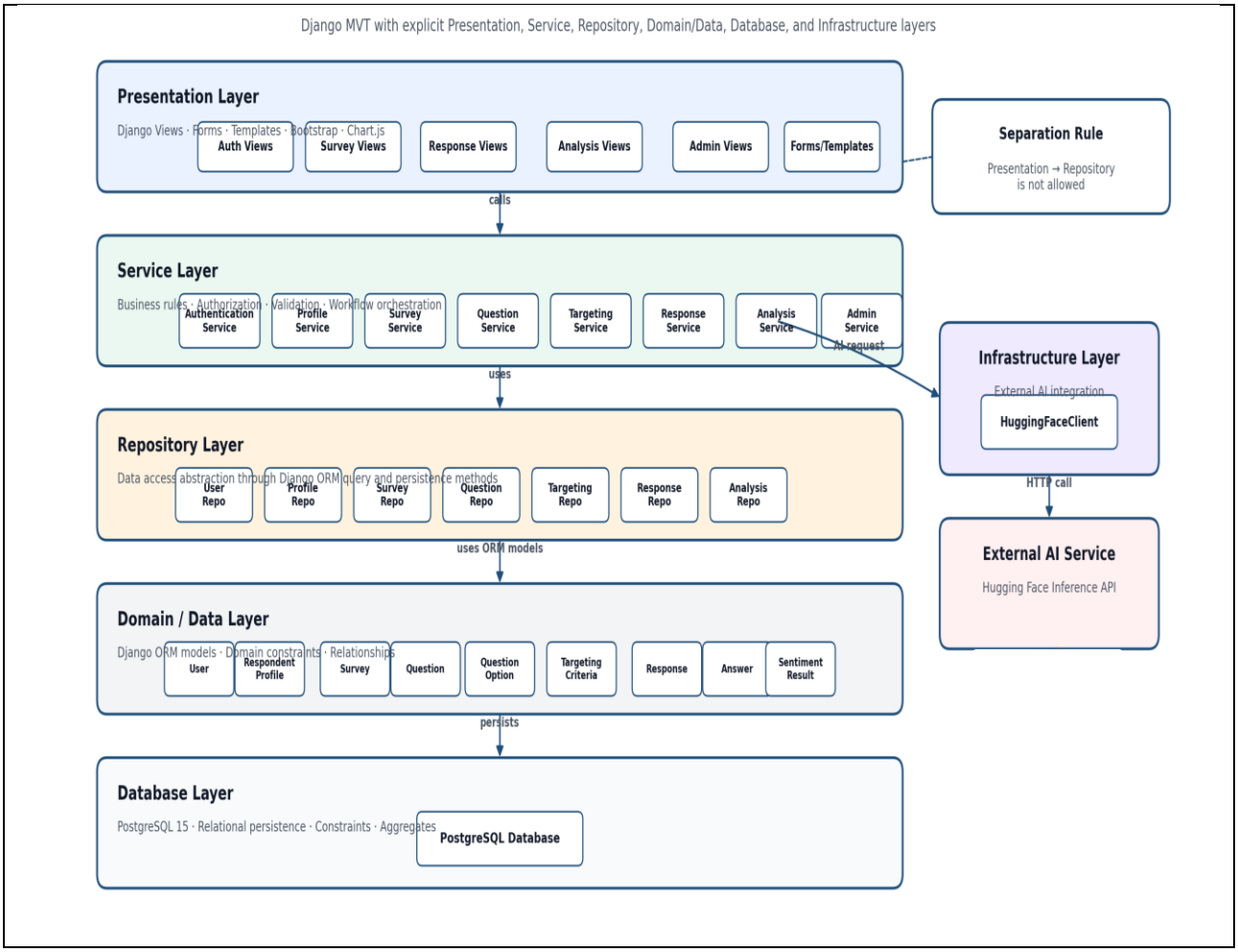

Figure 5.2: Layered System Architecture Design

5.1.3 System Architecture Design

The system architecture is organized into five main layers. The presentation layer handles HTTP requests, form validation, template rendering, and user interface responses. It does not contain business rules. The service layer contains the core business logic, including role validation, survey lifecycle rules, targeting checks, duplicate response prevention, and sentiment analysis orchestration. The repository layer isolates database queries and persistence operations. The domain/data layer defines the Django ORM models, relationships, constraints, and helper methods. The infrastructure layer isolates external communication with the Hugging Face Inference API.

Table 5.1: Architectural Layers

Layer	Implementation Location	Responsibility
Presentation Layer	`apps/*/views.py`, `apps/*/forms.py`, `templates/`, `static/`	Handles requests, forms, templates, UI rendering, and user feedback.
Service Layer	`services/`	Contains business rules, authorization checks, validation, and workflow orchestration.
Repository Layer	`repositories/`	Encapsulates database queries and persistence operations through Django ORM.
Domain / Data Layer	`apps/core/models/`	Defines persistent entities, relationships, constraints, and domain helper methods.
Infrastructure Layer	`infrastructure/huggingface_client.py`	Handles external AI communication with the Hugging Face Inference API.
Database Layer	PostgreSQL	Stores users, profiles, surveys, questions, responses, answers, and sentiment analysis results.

5.2 Database Design

5.2.1 Chen Entity Relationship Diagram

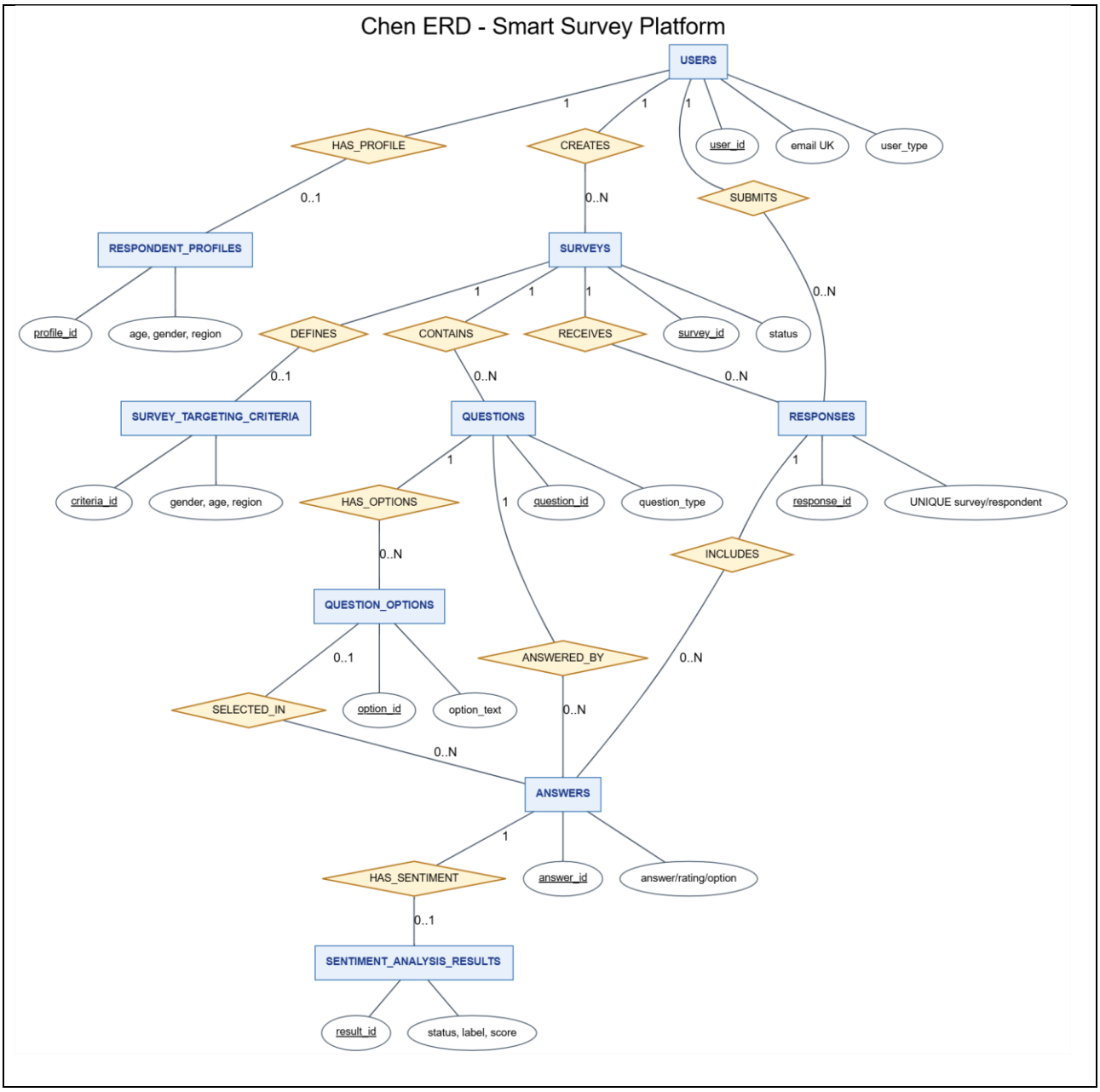

Figure 5.3: Chen Entity Relationship Diagram

5.2.2 Relational Schema

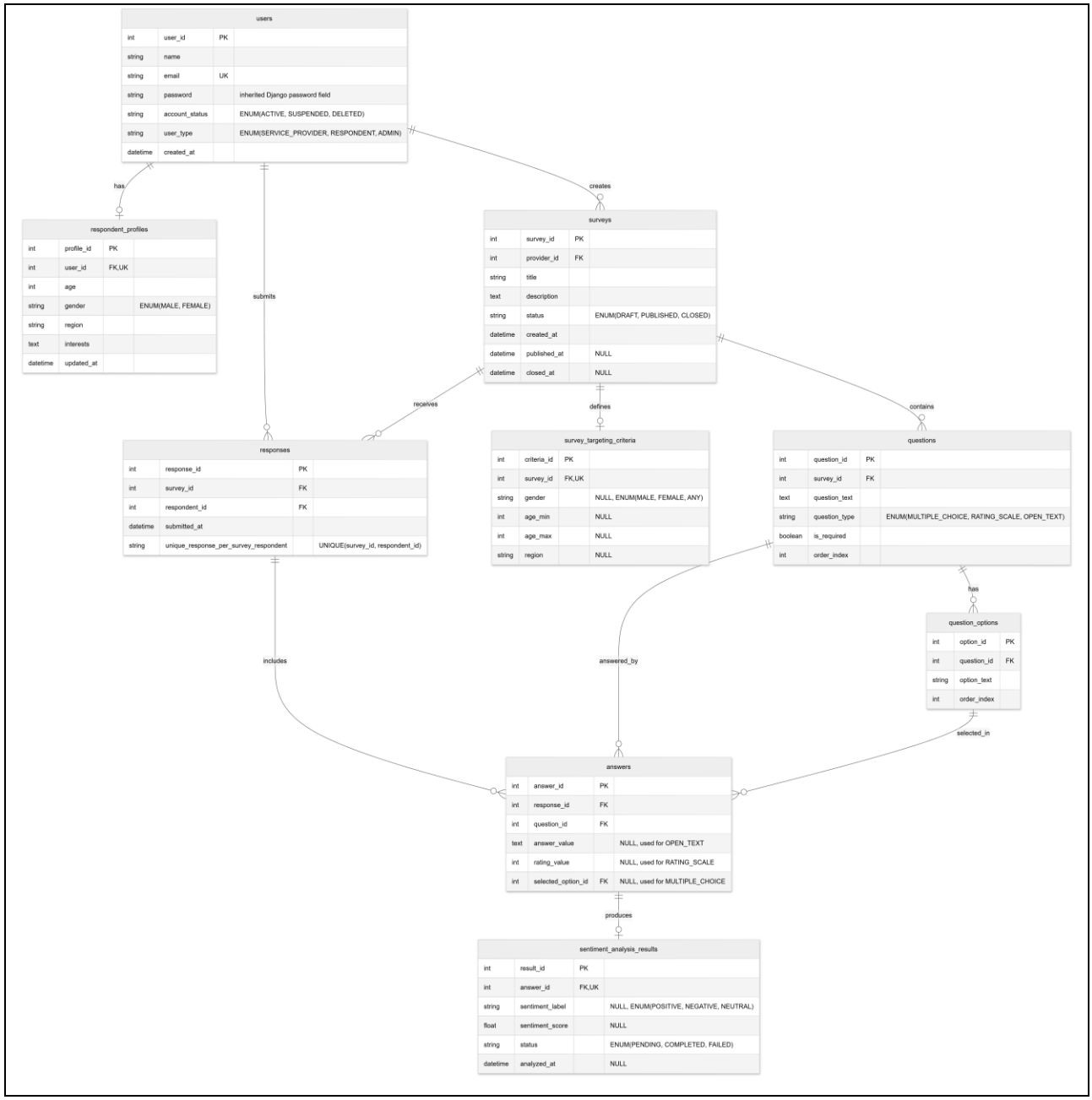

Figure 5.4: Relational Schema

Table 5.2: Database Entities

Model	Database Table	Purpose
User	`users`	Custom user model using email authentication, role, account status, and creation timestamp.
RespondentProfile	`respondent_profiles`	One-to-one demographic profile for respondent users, including age, gender, region, interests, and update timestamp.
Survey	`surveys`	Survey owned by a service provider, with title, description, status, creation, publication, and closing timestamps.
Question	`questions`	Survey question with question text, question type, required flag, and ordering index.
QuestionOption	`question_options`	Multiple-choice option linked to a question.
TargetingCriteria	`survey_targeting_criteria`	One-to-one survey targeting configuration using gender, age range, and region.
Response	`responses`	Survey submission by a respondent, with a uniqueness rule preventing duplicate participation.
Answer	`answers`	Question-level answer supporting open-text values, rating-scale values, and selected multiple-choice options.
Sentiment Analysis Result	`sentiment_analysis_results`	One-to-one sentiment analysis result for open-text answers, including label, score, status, and analysis timestamp.

5.2.3 Constraints and Keys

Table 5.3: Database Constraints and Keys

Key / Constraint	Type	Purpose
`users.user_id`	Primary Key	Uniquely identifies each user account.
`users.email`	Unique	Prevents duplicate accounts and supports email-based authentication.
`respondent_profiles.profile_id`	Primary Key	Uniquely identifies each respondent profile.
`respondent_profiles.user_id`	One-to-One Foreign Key	Ensures that each respondent has at most one demographic profile.

Key / Constraint	Type	Purpose
surveys.survey_id	Primary Key	Uniquely identifies each survey.
surveys.provider_id	Foreign Key	Links each survey to its service provider.
questions.question_id	Primary Key	Uniquely identifies each survey question.
questions.survey_id	Foreign Key	Links each question to its survey.
question_options.option_id	Primary Key	Uniquely identifies each multiple-choice option.
question_options.question_id	Foreign Key	Links each option to its question.
survey_targeting_criteria.criteria_id	Primary Key	Uniquely identifies each targeting criteria record.
survey_targeting_criteria.survey_id	One-to-One Foreign Key	Ensures that each survey has at most one targeting criteria record.
responses.response_id	Primary Key	Uniquely identifies each submitted response.
responses.survey_id	Foreign Key	Links each response to the survey being answered.
responses.respondent_id	Foreign Key	Links each response to the respondent who submitted it.
responses(survey_id, respondent_id)	Unique Constraint	Prevents duplicate survey participation by the same respondent.
answers.answer_id	Primary Key	Uniquely identifies each answer.
answers.response_id	Foreign Key	Links each answer to its submitted response.
answers.question_id	Foreign Key	Links each answer to the question being answered.
answers.selected_option_id	Nullable Foreign Key	Supports multiple-choice answers without forcing selected options for open-text or rating-scale questions.
sentiment_analysis_results.result_id	Primary Key	Uniquely identifies each sentiment analysis result.
sentiment_analysis_results.answer_id	One-to-One Foreign Key	Ensures that each analyzed answer has at most one sentiment analysis result.

5.3 External Interface Design

The Smart Survey Platform is implemented as a server-rendered Django web application. The current system does not expose a public REST API. User interactions are handled through Django URL routes, views, forms, and templates. The only external API integration in the current implementation is the Hugging Face Inference API, which is used for sentiment analysis of open-text answers.

Table 5.4: External API Design

Field	Description
API Name	Hugging Face Inference API
API Type	External AI API
Purpose	Perform sentiment analysis on open-text survey answers.
Path / Endpoint	`HF_SENTIMENT_MODEL_URL`
Method	`POST`
Authentication	Bearer token configured through `HF_API_TOKEN` environment variable.
Parameters	`text`: the open-text answer submitted by the respondent.
Request Body	`{ "inputs": text }`
Return Value	Sentiment prediction result containing label and confidence score.
Normalized Return Value	`POSITIVE`, `NEUTRAL`, or `NEGATIVE` with a numeric score.
Success Handling	The sentiment analysis result is stored with status `COMPLETED`.
Error Codes / Error Cases	`401/403`: invalid or missing token.`404`: invalid model endpoint.`429`: rate limit.`5xx`: external service failure.Timeout, request failure, invalid JSON, invalid payload, unknown label, or invalid score are handled as external analysis errors.
Failure Handling	The submitted response remains saved, and the sentiment analysis result is stored with status `FAILED`.
Implementation Location	`infrastructure/huggingface_client.py`
Security Note	The API token is not exposed to templates or client-side JavaScript.

5.4 Security and Authorization Design

Security and authorization are implemented using Django authentication, session management, role-based access control, service-layer validation, ownership checks, database constraints, and environment-based configuration. The system uses Django session-based authentication rather than JWT or OAuth.

Table 5.5: Security and Authorization Design

Security Control	Design Decision
Authentication Mechanism	The system uses Django session-based authentication with email and password.
Password Storage	Passwords are stored using Django password hashing, not plaintext storage.
Session Management	Login and logout are handled through Django authentication and session infrastructure.
CSRF Protection	Django CSRF protection is used for form submissions.
Secret Management	Runtime secrets such as database credentials and Hugging Face API token are loaded from environment variables.
Role-Based Access Control	The system separates access according to Service Provider, Respondent, System Administrator, and Guest roles.
Ownership Validation	Service providers can update, delete, publish, close, and view analytics only for their own surveys.
Respondent Eligibility	Respondents can view and answer only published surveys matching their demographic profile and not already answered.
Duplicate Response Prevention	Service-level validation and a database unique constraint prevent repeated participation in the same survey.
External API Protection	Hugging Face API access is isolated in `HuggingFaceClient`; API tokens are not exposed to templates or JavaScript.
Admin Self-Protection	Administrators cannot suspend or soft-delete their own account.
Soft Deletion	User deletion is handled through account status rather than immediate physical deletion.

Table 5.6: User Roles and Access Levels

Role	Access Level
Guest	Can access public pages, registration pages, and login page only.
Service Provider	Can manage own surveys, questions, targeting criteria, publication/closing, and survey analytics.
Respondent	Can manage demographic profile, view eligible surveys, submit responses, and view answered surveys.
System Administrator	Can manage user account status and generate system reports.

Section Four: Implementation and Testing

Chapter 6: Implementation Environment and Tools

6.1 Software Tools and Libraries

Table 6.1: Technology Stack

Tool	Purpose	Evidence / Status
Python	Main backend programming language.	Confirmed by requirements.txt, manage.py, and .github/workflows/django-ci.yml. CI uses Python 3.11.
Django	Web application framework.	Confirmed by requirements.txt and config/settings.py.
PostgreSQL	Relational database server.	Confirmed by config/settings.py and docker-compose.yml.
Docker Compose	Runs the local PostgreSQL service.	Confirmed by docker-compose.yml. It does not define a Django application service.
Git	Version control tool.	Confirmed by Git repository status and branch information.
GitHub	Remote source control hosting.	Confirmed by git remote -v: https://github.com/Yaman8almatar/smart-survey-platform.git.
GitHub Actions	Continuous Integration tool.	Confirmed by .github/workflows/django-ci.yml; CI only, not deployment.
pytest	Test runner for service-layer tests.	Confirmed by requirements.txt, pytest.ini, and tests/services/.
pytest-django	Django integration for pytest.	Confirmed by requirements.txt and pytest.ini.
Hugging Face Inference API	External sentiment analysis service.	Confirmed by infrastructure/huggingface_client.py and .env.example.
Developer editor	Code editing environment.	Team-reported / not repository-evidenced. No PyCharm, VS Code, or other editor is claimed.

6.2 Development Environment and Project Structure

The project is a server-rendered Django web application organized using a layered architecture. The repository separates Django views and templates from services, repositories, domain models, and external infrastructure clients.

Table 6.2: Development Environment and Project Structure

Item	Description
Development environment	Local inspection was performed on Windows (Microsoft Windows NT 10.0.26200.0). CI is configured separately on Ubuntu.
Operating system	Local OS confirmed as Windows. CI runner confirmed as ubuntu-latest. Production OS is not confirmed.
Application type	Web-based Django survey management platform, confirmed by README.md, config/settings.py, and templates/base.html.
Main configuration folder	config/ contains Django settings and URL configuration.
Django apps	apps/admin_panel/, apps/analysis/, apps/core/, apps/responses/, apps/surveys/, and apps/users/.
Domain models	apps/core/models/ contains User, RespondentProfile, Survey, Question, QuestionOption, TargetingCriteria, Response, Answer, and SentimentAnalysisResult.
Services	services/ contains business workflow classes such as authentication, survey, question, targeting, response, analysis, profile, and admin services.
Repositories	repositories/ contains data access classes using Django ORM.
Infrastructure	infrastructure/huggingface_client.py isolates Hugging Face API communication.
Templates	Shared templates are under templates/; app templates are under apps/*/templates/.
Static files	static/css/site.css and static/js/site.js provide project CSS and JavaScript.
Tests	tests/services/ contains 8 service test files and 160 detected test functions.
Documentation/report assets	Docs_1/ contains seminar/report assets. docs/ is local ignored working context according to .gitignore.

6.3 Programming Languages and Frameworks

Table 6.3: Programming Languages and Frameworks

Category	Technology	Purpose
Programming language	Python	Backend application code and Django project execution.
Web framework	Django 4.2.11	Web routing, views, models, forms, authentication, and templates.
Template engine	Django Templates	Server-rendered HTML templates.
UI framework	Bootstrap 5.3.3	User interface styling, confirmed in templates/base.html.
Icon library	Bootstrap Icons 1.11.3	Navigation and UI icons, confirmed in templates/base.html.
Charting library	Chart.js 4.4.1	Dashboard and report charts, confirmed in templates/base.html.
ORM	Django ORM	Model mapping and database access.
HTTP client	requests 2.31.0	External Hugging Face API calls in infrastructure/huggingface_client.py.
External AI API	Hugging Face Inference API	Sentiment analysis for open-text answers.
Test framework	pytest 8.1.1	Service-layer test execution.
Django test integration	pytest-django 4.8.0	Django settings and database support for pytest.

6.4 Database Server and Operating System

Table 6.4: Database Server and Runtime Environment

Item	Technology / Configuration	Purpose
Database server	PostgreSQL 15 from docker-compose.yml (postgres:15).	Local relational database service.
Database driver	psycopg[binary]==3.1.18 from requirements.txt.	Connect Django to PostgreSQL.
ORM	Django ORM from Django 4.2.11.	Map Django models to PostgreSQL tables.
Database configuration	config/settings.py reads DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, and DB_PORT.	Environment-based database configuration.

Item	Technology / Configuration	Purpose
Local operating system	Windows confirmed by local inspection command.	Local development/inspection environment.
CI operating system	ubuntu-latest in .github/workflows/django-ci.yml.	GitHub Actions test runner.
Django development server	python manage.py runserver documented in README.md.	Local development server.

6.5 Dependencies

Table 6.5: Main Python Dependencies

Dependency	Version	Purpose
Django	4.2.11	Web framework, ORM, templates, forms, authentication, and settings.
psycopg[binary]	3.1.18	PostgreSQL database driver.
python-dotenv	1.0.1	Load environment variables from .env.
pytest	8.1.1	Python test runner.
pytest-django	4.8.0	Django integration for pytest.
requests	2.31.0	HTTP client for external Hugging Face API calls.

6.6 Version Control and Source Control Tools

Table 6.6: Version Control and Source Control Tools

Tool / Item	Purpose / Status
Git	Repository uses Git for source control. Current branch status: main...origin/main.
GitHub remote	Confirmed remote: https://github.com/Yaman8almatar/smart-survey-platform.git.
Current branch	main is the active local branch.
Branch structure	Confirmed local branches include main, develop, feature/backend-core, and feature/frontend. Confirmed remote branches include origin/main, origin/develop, origin/feature/ai-analysis, origin/feature/backend-core, and origin/feature/frontend.
Tracked files	git ls-files reported 177 tracked files during inspection.
.gitignore	Ignores .env, virtual environments, cache files, local Codex files, and temporary Office files.
CI workflow tracking status	.github/workflows/django-ci.yml exists locally; current Git status showed .github/ as untracked before commit.

6.7 CI/CD Tools

Table 6.7: Continuous Integration and Deployment Status

Item	Status
CI tool	GitHub Actions.
Workflow file	.github/workflows/django-ci.yml.
Workflow name	Django CI.
Trigger	push and pull_request for main and develop.
Runner	ubuntu-latest.
Python version	Python 3.11 through actions/setup-python@v5.
Database service	PostgreSQL 15 through postgres:15.
Dependency installation	python -m pip install --upgrade pip and python -m pip install -r requirements.txt.
CI commands	python manage.py check and python -m pytest tests/services -q.
External API secrets	Uses dummy Hugging Face values in CI. No real token is stored in the repository.
Continuous Deployment	Not implemented. No deployment workflow exists.
Production Deployment	Not claimed. No production deployment evidence exists.
Execution evidence	CI workflow is configured; successful execution evidence should be captured from GitHub Actions after pushing.

6.8 Testing Environment

Table 6.8: Testing Environment

Item	Description
Test framework	pytest==8.1.1 from requirements.txt.
Django test support	pytest-django==4.8.0 from requirements.txt; pytest.ini sets DJANGO_SETTINGS_MODULE = config.settings.
Test location	tests/services/.
Service test files	8 files were detected under tests/services/.
Test functions	160 def test_ functions were detected under tests/services/.
Django check command	python manage.py check.
pytest command	python -m pytest tests/services -q.

Item	Description
Test scope	Service-layer workflows, validation rules, repositories through Django ORM, and AI analysis behavior using test doubles.
External API testing rule	CI uses dummy Hugging Face values; service tests should not depend on a real external API call.
Local execution evidence	Not confirmed in this chapter generation step. Local python command was unavailable; py --version returned Python 3.7.8, while CI is configured for Python 3.11.
GitHub Actions evidence	CI workflow is configured; successful run evidence should be captured after pushing the workflow to GitHub.

Chapter 7: Software Implementation

This chapter presents representative implementation examples from the Smart Survey Platform. The full source code is kept in the repository and can be placed in an appendix if required. The selected snippets demonstrate the implemented project structure, modules, web route handling, external API integration, data access, business rules, user interface, and AI-assisted sentiment analysis.

7.1 Project File Structure

The project uses a layered Django structure. The main folders are shown below.

Code snippet 7.1: Compact Project Structure

File path	Repository root
Purpose	Shows the main project folders without listing every implementation file.

```
smart-survey-platform/
|-- config/
|-- apps/
|   |-- core/
|   |   `-- models/
|   |-- users/
|   |-- surveys/
|   |-- responses/
|   |-- analysis/
|   `-- admin_panel/
|-- services/
|-- repositories/
|-- infrastructure/
|-- templates/
|-- static/
|-- tests/
|   `-- services/
|-- requirements.txt
|-- docker-compose.yml
`-- .github/
    `-- workflows/
```

Explanation: The structure separates configuration, Django apps, services, repositories, infrastructure clients, templates, static assets, tests, dependencies, database setup, and CI workflow files.

Table 7.1: Project File Structure

Folder / File	Responsibility
config/	Django settings, root URL routing, WSGI/ASGI configuration.
apps/core/models/	Persistent domain models such as users, surveys, questions, responses, answers, and sentiment results.
apps/users/	Registration, login, logout, home redirection, user forms, and user templates.
apps/surveys/	Provider survey creation, editing, deletion, question management, targeting, publishing, and closing.
apps/responses/	Respondent profile management, eligible surveys, response submission, and answered survey history.
apps/analysis/	Provider analytics dashboard views and templates.

Folder / File	Responsibility
apps/admin_panel/	Administrator dashboard, user management, and system reports.
services/	Business logic and workflow orchestration.
repositories/	Data access layer using Django ORM queries.
infrastructure/	External integration clients, including the Hugging Face client.
templates/	Shared base layout and common template structure.
static/	Project CSS and JavaScript files.
tests/services/	Service-layer test suite.
requirements.txt	Python dependencies.
docker-compose.yml	Local PostgreSQL service configuration.
.github/workflows/	GitHub Actions CI workflow.

7.2 Module Implementation

The application is organized by feature modules. Each module combines Django routes, views, forms, templates, service classes, repository classes, and domain models where required.

Table 7.2: Implementation Modules

Module	Main Files	Responsibility
Authentication and Users	apps/users/views.py; apps/users/forms.py; services/authentication_service.py; repositories/user_repository.py; apps/core/models/user.py	Register users, authenticate by email/password, reject inactive accounts, and route users by role.
Survey Management	apps/surveys/views.py; apps/surveys/forms.py; services/survey_service.py; repositories/survey_repository.py; apps/core/models/survey.py	Create, update, delete, publish, close, and list provider surveys.
Question Management	apps/surveys/views.py; apps/surveys/forms.py; services/question_service.py; repositories/question_repository.py; apps/core/models/question.py; apps/core/models/question_option.py	Add supported question types and options to draft surveys.
Targeting Criteria	apps/surveys/views.py; apps/surveys/forms.py; services/targeting_service.py; repositories/targeting_repository.py; apps/core/models/targeting_criteria.py	Define demographic restrictions and match respondents to surveys.
Response Submission	apps/responses/views.py; services/response_service.py; repositories/response_repository.py; apps/core/models/response.py; apps/core/models/answer.py	Validate eligibility, prevent duplicate responses, store responses, and store answers.
Analysis and Sentiment	apps/analysis/views.py; services/analysis_service.py; repositories/analysis_repository.py; infrastructure/huggingface_client.py; apps/core/models/sentiment_analysis_result.py	Build analytics and analyze open-text answers using Hugging Face.

Module	Main Files	Responsibility
Administration	apps/admin_panel/views.py; services/admin_service.py; repositories/user_repository.py; repositories/survey_repository.py; repositories/response_repository.py	Manage user accounts and generate system reports.
Core Domain Models	apps/core/models/*.py; core/enums.py; core/exceptions.py	Define persistent entities, status choices, validation helpers, and domain exceptions.
Targeting Criteria	apps/surveys/views.py; apps/surveys/forms.py; services/targeting_service.py; repositories/targeting_repository.py; apps/core/models/targeting_criteria.py	Define demographic restrictions and match respondents to surveys.
Response Submission	apps/responses/views.py; services/response_service.py; repositories/response_repository.py; apps/core/models/response.py; apps/core/models/answer.py	Validate eligibility, prevent duplicate responses, store responses, and store answers.
Analysis and Sentiment	apps/analysis/views.py; services/analysis_service.py; repositories/analysis_repository.py; infrastructure/huggingface_client.py; apps/core/models/sentiment_analysis_result.py	Build analytics and analyze open-text answers using Hugging Face.
Administration	apps/admin_panel/views.py; services/admin_service.py; repositories/user_repository.py; repositories/survey_repository.py; repositories/response_repository.py	Manage user accounts and generate system reports.

7.3 API Implementation

The project does not expose a public REST API. It is a server-rendered Django application that uses URL routes, HTML forms, and POST requests for web workflows. The only external API integration is the Hugging Face Inference API used for sentiment analysis.

7.3.1 Django Web Route and Form Submission Example

The respondent survey submission flow is implemented as a Django route and view. The route renders the response.

Table 7.3: Django Web Route Example

Item	Implementation
Route path	/responses/surveys/<survey_id>/submit/
HTTP methods	GET for the form and POST for submission

Item	Implementation
View function	submit_survey_response
Service called	ResponseService().submit_response(...)
Return behavior	On success, redirects to submission confirmation. On service errors, shows a message and keeps the user in the workflow.

Code snippet 7.2: Survey Response Route

File path	apps/responses/urls.py
Purpose	Defines the route used for respondent survey submission.

```
path(
    "surveys/<int:survey_id>/submit/",
    views.submit_survey_response,
    name="submit_survey_response",
),
```

Explanation: This URL pattern maps a survey response submission URL to the Django view that renders and processes the form.

Code snippet 7.3: Survey Response Submission Handler

File path	apps/responses/views.py
Purpose	Processes submitted response form data and delegates business logic to the service layer.

```
if request.method == "POST":
    try:
        ResponseService().submit_response(
            request.user,
            survey_id,
            _collect_answers(request.POST, questions),
        )
    except SmartSurveyException as error:
        _show_service_error(request, error)
    else:
        messages.success(request, "Survey response submitted successfully.")
        return redirect("responses:submission_confirmation")
```

Explanation: The view does not store answers directly. It collects form input, calls ResponseService, and returns either a success redirect or an error message.

7.3.2 External API Integration Example

The platform integrates with the Hugging Face Inference API through HuggingFaceClient. The API endpoint is read from HF_SENTIMENT_MODEL_URL, and the token is read from HF_API_TOKEN. The method sends a POST request with {"inputs": text} and returns a normalized sentiment label and score.

Table 7.4: External API Integration

Item	Implementation
Endpoint source	HF_SENTIMENT_MODEL_URL environment variable
Method	POST

Item	Implementation
Authentication	Bearer token from HF_API_TOKEN
Request body	{"inputs": text}
Return value	Normalized sentiment label and score
Error handling	Raises ExternalAnalysisServiceError for missing config, invalid input, failed request, or invalid response

Code snippet 7.4: Hugging Face HTTP Request

File path	infrastructure/huggingface_client.py
Purpose	Sends open-text answers to the external Hugging Face endpoint without exposing the token value.

```
headers = {"Authorization": f"Bearer {self.api_token}"}
payload = {"inputs": text}

try:
    response = requests.post(
        self.model_url,
        headers=headers,
        json=payload,
        timeout=self.timeout,
    )
    response.raise_for_status()
except requests.RequestException as exc:
    raise ExternalAnalysisServiceError(
        "Hugging Face request failed."
    ) from exc
```

Explanation: The infrastructure client hides external HTTP communication from the service layer. The code uses environment-configured values but does not hardcode a real token.

7.4 Data Access Layer Implementation

Table 7.5: Repository Classes

Repository	**Models/Tables Accessed**	**Responsibility**
UserRepository	User / users	CRUD, email lookup, user listing, and account count queries.
ProfileRepository	RespondentProfile / respondent_profiles	Profile save, lookup, and update.
SurveyRepository	Survey / surveys	Survey CRUD, provider listing, published survey lookup, status counts.
QuestionRepository	Question, QuestionOption	Question CRUD and option lookup/save operations.
TargetingRepository	TargetingCriteria	Criteria save, lookup, update, and create-or-update operations.
ResponseRepository	Response, Answer	Response save, duplicate checks, answer persistence, and response aggregates.
AnalysisRepository	SentimentAnalysisResult	Sentiment result save, lookup, update, and survey-linked retrieval.

Code snippet 7.5: Duplicate Response Check

File path	repositories/response_repository.py
Purpose	Checks whether a respondent already answered a survey.

```
def has_response_for_survey(self, survey_id, respondent_id):
    """Return whether a respondent already answered a survey."""
    return Response.objects.filter(
        survey_id=survey_id,
        respondent_id=respondent_id,
    ).exists()
```

Explanation: This repository method supports duplicate response prevention by asking the database whether a matching response already exists.

Code snippet 7.6: Create or Update Targeting Criteria

File path	repositories/targeting_repository.py
Purpose	Creates or updates survey targeting criteria using Django ORM.

```
criteria, _created = TargetingCriteria.objects.update_or_create(
    survey=survey,
    defaults={
        "gender": gender,
        "age_min": age_min,
        "age_max": age_max,
        "region": region,
    },
)
return criteria
```

Explanation: The repository provides one method for both creating and updating demographic targeting data for a survey.

Code snippet 7.7: Most Active Surveys Query

File path	repositories/survey_repository.py
Purpose	Produces an aggregate query for the most active surveys.

```
return (
    Survey.objects.annotate(response_count=Count("responses"))
    .filter(response_count__gt=0)
    .order_by("-response_count", "survey_id")[:limit]
)
```

Explanation: This ORM query is used by administrator reporting to rank surveys by submitted response count.

7.5 Business Logic Layer Implementation

Business rules are implemented in service classes rather than directly inside views. This makes workflows easier to test and keeps the view layer focused on HTTP behavior.

Table 7.6: Service Classes

Service	Responsibility
AuthenticationService	Registration, duplicate email validation, password hashing, and login credential validation.
ProfileService	Respondent demographic profile retrieval and update.
SurveyService	Survey lifecycle, ownership checks, publication rules, close rules, and eligible survey listing.
QuestionService	Question creation, option handling, question validation, and response form question data.
TargetingService	Targeting criteria save/update and respondent-to-survey matching.
ResponseService	Eligibility validation, duplicate prevention, answer validation, response persistence, and sentiment trigger.
AnalysisService	Open-text sentiment analysis and survey analytics summaries.
AdminService	Account activation/suspension/soft deletion and system report generation.

Code snippet 7.8: Survey Publication Rules

File path	services/survey_service.py
Purpose	Enforces survey publication rules before changing survey status.

```
def publish_survey(self, provider_user, survey_id):
    survey = self._get_owned_survey(provider_user, survey_id)


    if not survey.can_edit():
        raise NotEditable("Only draft surveys can be published.")


    if not self.question_repository.find_by_survey_id(
        survey.survey_id
    ).exists():
        raise ValidationError("A survey must have at least one question.")


    if self.targeting_repository.find_by_survey_id(survey.survey_id) is None:
        raise TargetingCriteriaNotFound(
            "Survey targeting criteria is required."
        )


    survey.publish()
    return self.survey_repository.update(survey)
```

Explanation: The service verifies ownership, draft status, question existence, and targeting criteria before publishing. This supports the publish survey use case.

Code snippet 7.9: Duplicate Prevention and Analysis Trigger

File path	services/response_service.py
Purpose	Prevents duplicate responses and triggers sentiment analysis after open-text answers are stored.

```
if self.validate_duplicate_response(respondent_user, survey_id):
    raise DuplicateResponse(
        "Respondent already submitted this survey."
    )

response = self.response_repository.save(
    Response(survey=survey, respondent=respondent_user)
)

if response.has_open_text_answers():
    self.analysis_service.analyze_open_text_answers(
        response.response_id
    )
```

Explanation: The response workflow first blocks repeat submissions, then saves the response, and only then triggers sentiment analysis for open-text answers.

Code snippet 7.10: Sentiment Failure Handling

File path	services/analysis_service.py
Purpose	Handles external sentiment failures without deleting the saved response.

```
try:
    analysis = self.huggingface_client.analyze_sentiment(
        answer.answer_value
    )
    label = self._map_external_label(analysis.get("label"))
    if label is None:
        raise ExternalAnalysisServiceError(
            "Unknown sentiment label."
        )

    result.mark_completed(label, analysis.get("score"), timezone.now())
except ExternalAnalysisServiceError:
    result.mark_failed()
```

Explanation: The service records completed sentiment results when the API succeeds and marks the result as failed when the external analysis fails.

7.6 User Interface Implementation

The user interface is implemented using server-rendered Django templates. Bootstrap provides styling and responsive components. Chart.js is used for analytics and administrator report charts. The base layout provides role-specific navigation for service providers, respondents, and administrators.

Table 7.7: User Interface Components

UI Component	Template / Static File	Purpose
Base layout	templates/base.html	Shared page shell, navigation, Bootstrap, Bootstrap Icons, Chart.js, messages, and static assets.
Provider dashboard	apps/surveys/templates/surveys/provider_dashboard.html	Shows provider survey metrics and recent surveys.
Respondent dashboard	apps/responses/templates/responses/respondent_dashboard.html	Shows respondent profile and eligible survey summary.
Eligible surveys page	apps/responses/templates/responses/eligible_surveys.html	Lists surveys the respondent is eligible to answer.
Submit response page	apps/responses/templates/responses/submit_survey_response.html	Renders open-text, rating-scale, and multiple-choice inputs.
Analytics dashboard	apps/analysis/templates/analysis/analytics_dashboard.html	Shows provider analytics, sentiment summary, and question results.
Admin dashboard	apps/admin_panel/templates/admin_panel/admin_dashboard.html	Provides administrator entry point.
User management	apps/admin_panel/templates/admin_panel/user_management.html	Supports activate, suspend, and soft-delete actions.
System reports	apps/admin_panel/templates/admin_panel/system_reports.html; static/js/site.js	Shows platform report charts using Chart.js.
Styling	static/css/site.css	Project visual styles.
JavaScript	static/js/site.js	Question option builder and system report charts.

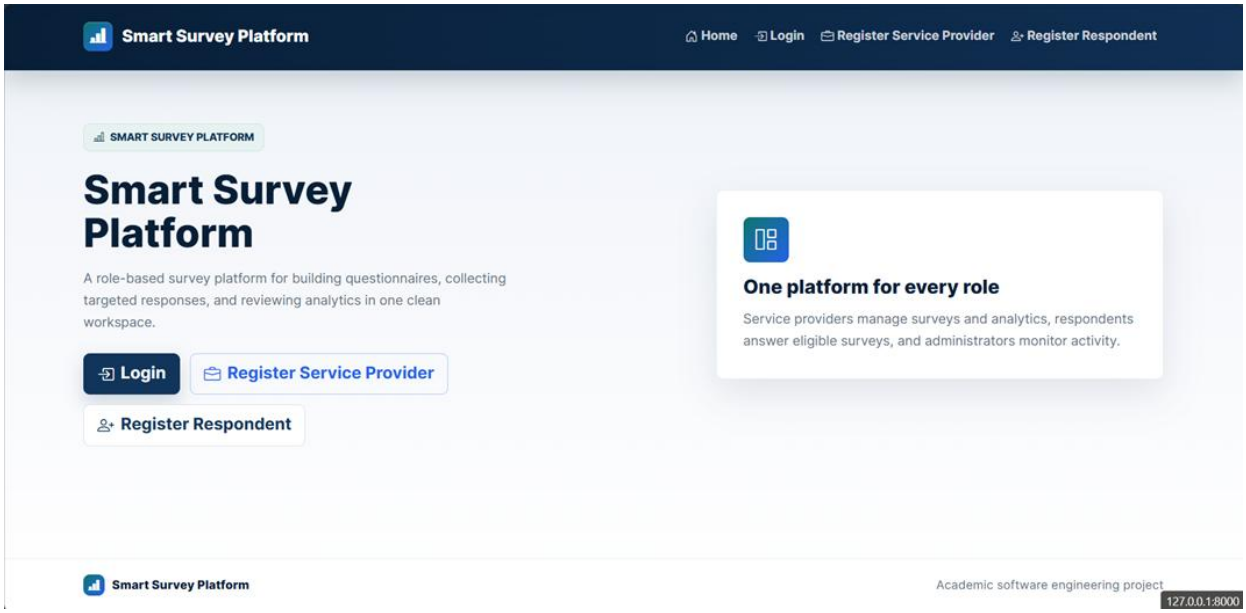

Figure 7.1: Public Home Page

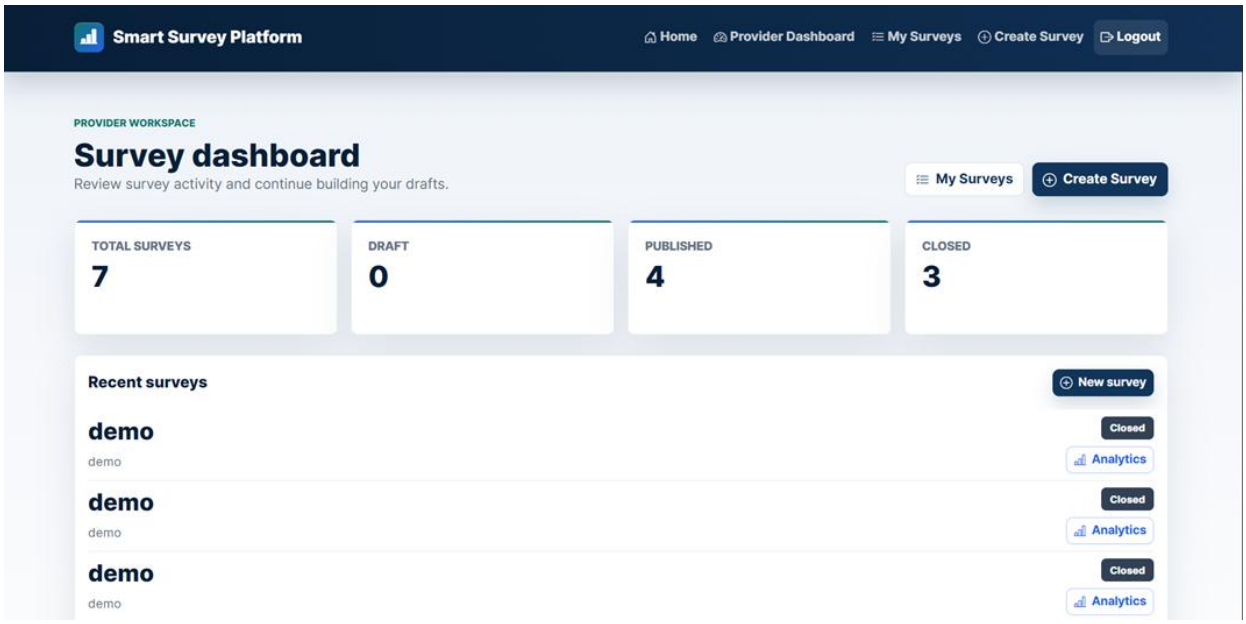

Figure 7.2: Provider Dashboard

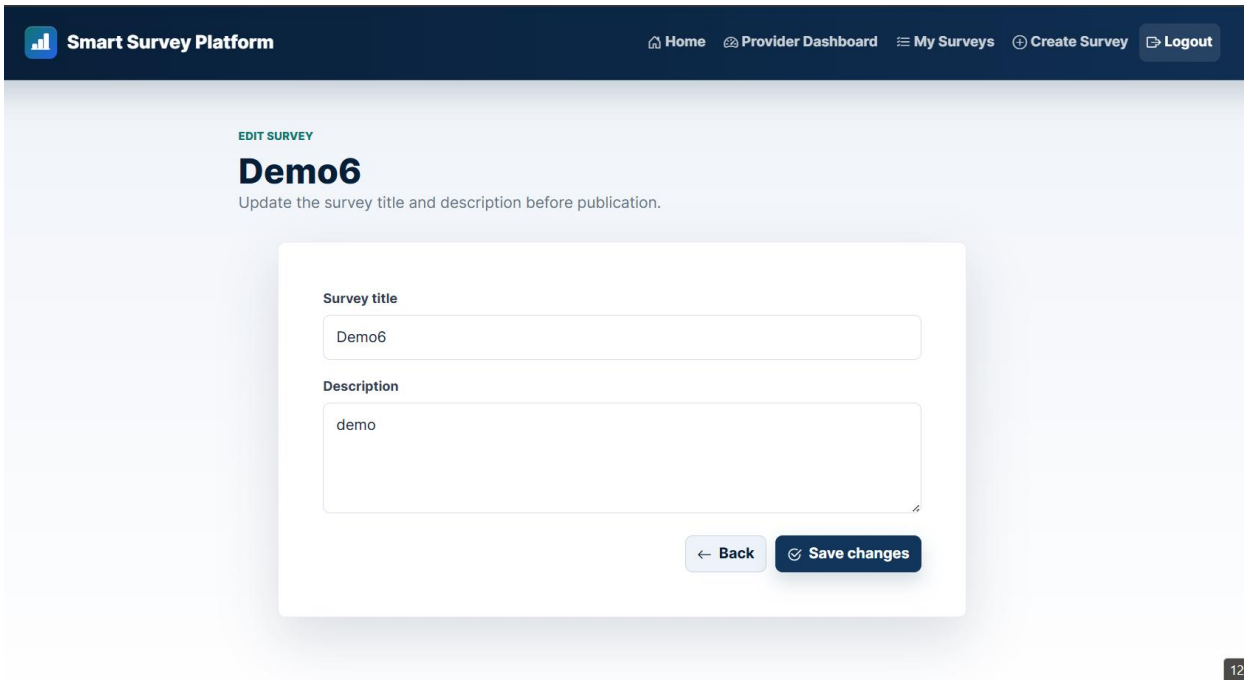

Figure 7.3: Survey Edit Page

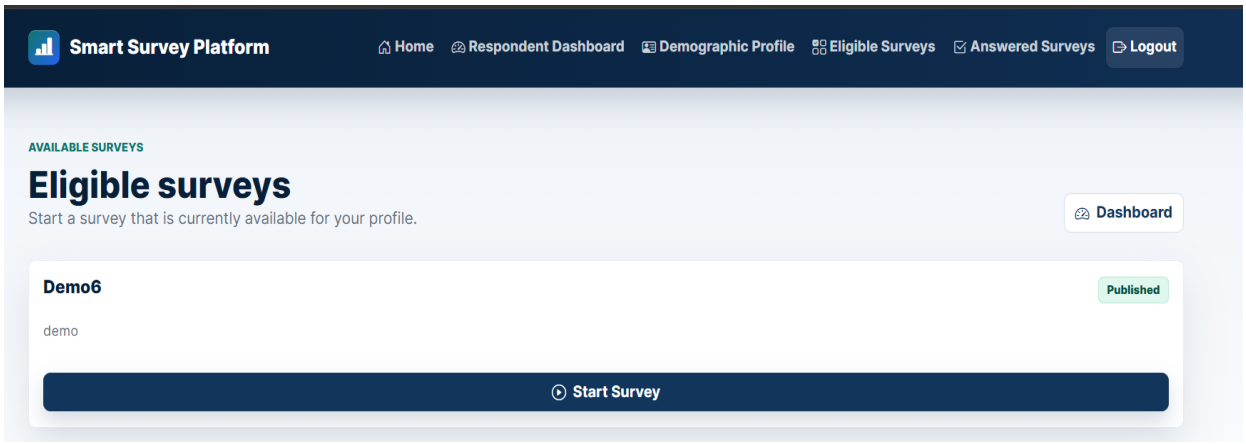

Figure 7.4: Eligible Surveys Page

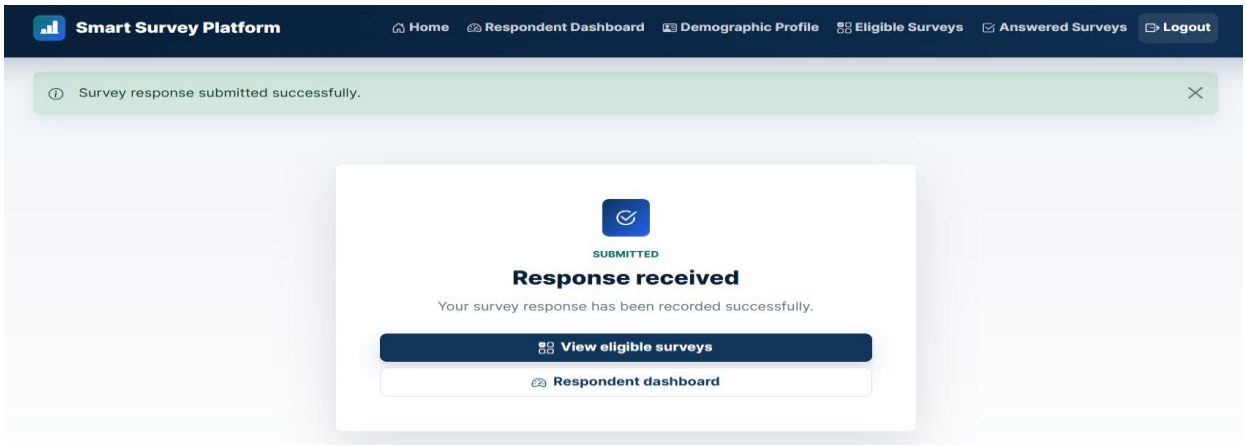

Figure 7.5: Submission Confirmation Page

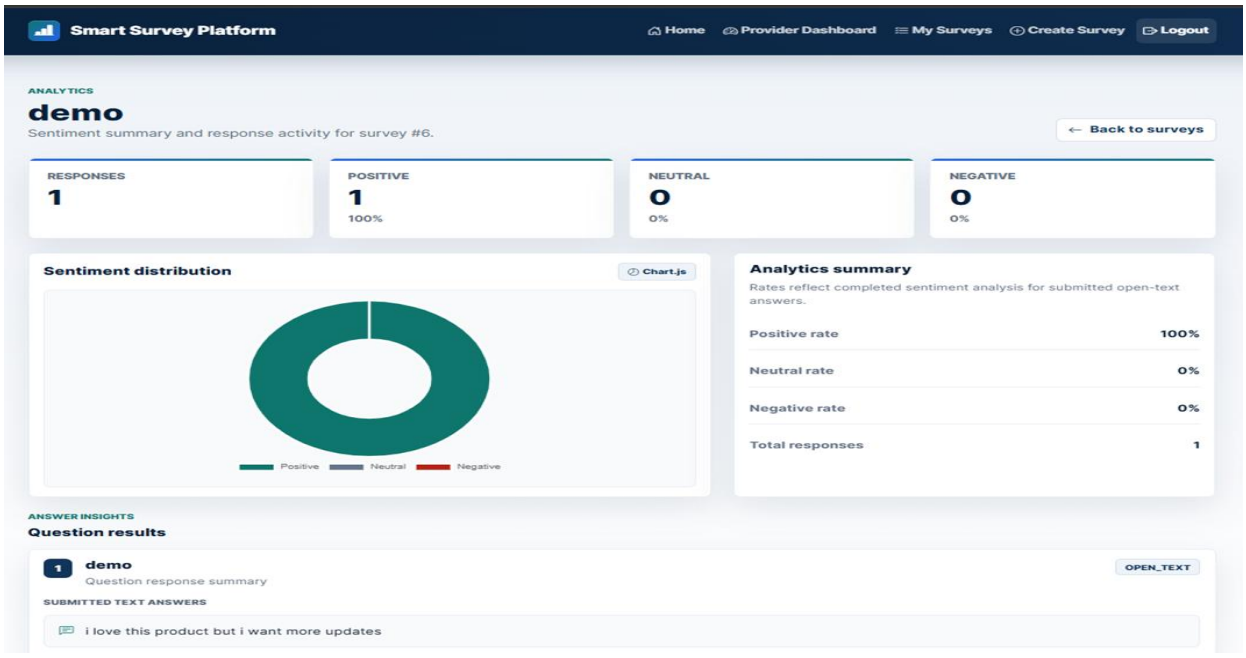

Figure 7.6: Analytics Dashboard

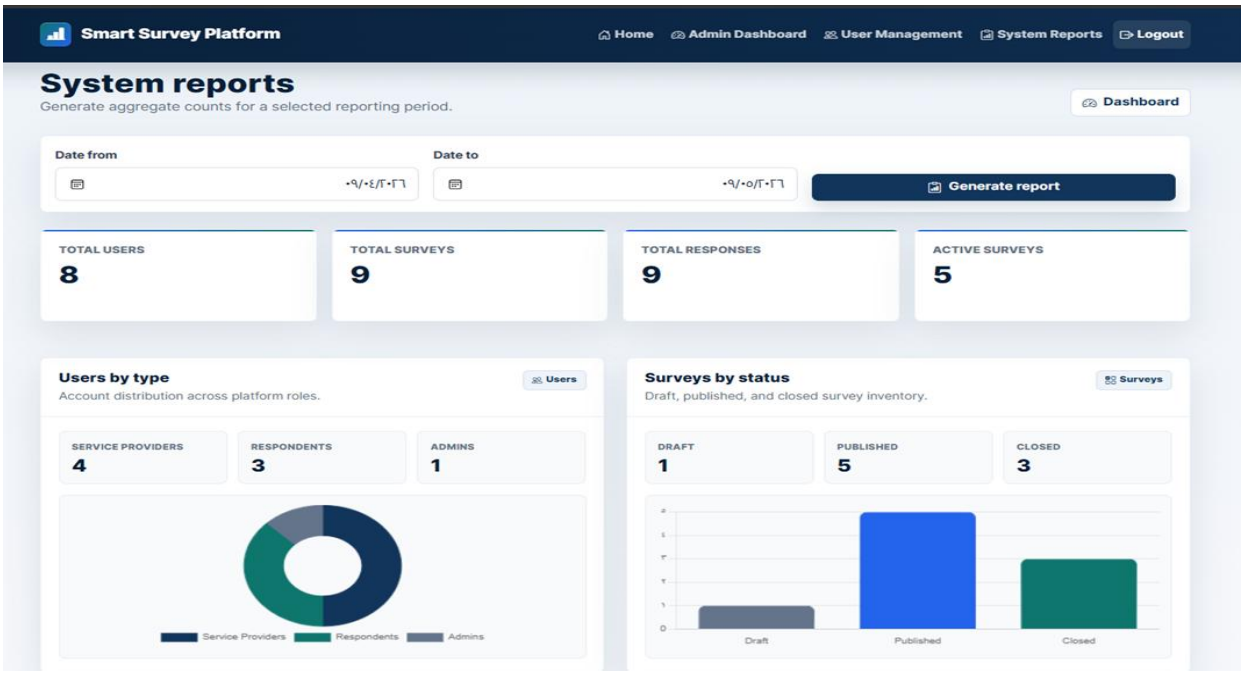

Figure 7.7: Admin System Reports Page

Code snippet 7.11: Dynamic Response Inputs

File path	apps/responses/templates/responses/submit_survey_response.html
Purpose	Renders different input controls based on each question type.

```
{% if question.is_open_text %}
    <textarea
        class="form-control"
        name="{{ question.answer_name }}"
        rows="4"
    ></textarea>
{% endif %}

{% if question.is_rating_scale %}
    <select
        class="form-select response-rating"
        name="{{ question.rating_name }}"
    >
```

Explanation: The template receives question metadata from the service layer and displays the correct input control for open-text and rating-scale questions. Multiple-choice inputs are rendered in the same template using radio buttons.

7.7 Intelligent Algorithm Implementation

The smart component is AI-assisted sentiment analysis for open-text answers. The implementation does not train a model from scratch. It stores the respondent response first, then analyzes open-text answers through the Hugging Face Inference API. Completed analysis stores a label, score, and analysis timestamp. Failed analysis marks the sentiment result as FAILED without deleting the submitted response.

Table 7.8: Smart Sentiment Analysis Flow

Step	Description	Output
1. Detect open-text answers	Response.has_open_text_answers() identifies whether the saved response contains open-text answers.	Decision to trigger analysis.
2. Create or reset pending analysis result	AnalysisService creates a pending SentimentAnalysisResult or resets an existing one.	Pending sentiment result.
3. Send text to HuggingFaceClient	AnalysisService calls HuggingFaceClient.analyze_sentiment(...).	External API request.
4. Normalize returned label and score	HuggingFaceClient maps provider labels to internal sentiment labels and converts score to a float.	Normalized label and score.
5. Store COMPLETED result	On success, mark_completed(...) stores label, score, and timestamp.	Completed sentiment result.
6. Store FAILED result if external API fails	On ExternalAnalysisServiceError, mark_failed() clears label/score and records failure status.	Failed sentiment result without losing the response.

Code snippet 7.12: Open-Text Answer Detection

File path	services/analysis_service.py
Purpose	Runs sentiment analysis only for answers that require it.

```
for answer in answers:
    if not answer.requires_sentiment_analysis():
        continue
    result = self._get_or_create_pending_result(answer)
```

Explanation: The service skips non-text answers and prepares a sentiment result record only for open-text answers.

Code snippet 7.13: Sentiment Label Normalization

File path	infrastructure/huggingface_client.py
Purpose	Normalizes external sentiment labels into internal sentiment categories.

```
def _normalize_label(self, label):
    """Normalize external sentiment labels into internal sentiment categories."""
    normalized_label = str(label).strip().upper()
    mapped_label = self.LABEL_MAP.get(normalized_label)
    if mapped_label is None:
        raise ExternalAnalysisServiceError(
            "Unknown sentiment label."
        )
    return mapped_label
```

Explanation: The external API can return different label formats. This method converts supported external labels into the platform internal sentiment categories.

The implementation uses an external pretrained sentiment model through an API; it does not train a new machine learning model.

Chapter 8: Testing and Evaluation

8.1 Test Plan

The test plan covers authentication, survey management, question handling, targeting criteria, respondent eligibility, response submission, duplicate prevention, sentiment analysis, and administrator functions. Automated testing is implemented mainly at the service layer using pytest and pytest-django. Manual validation is still required for browser-based UI behavior, while GitHub Actions is used to verify the automated test suite in CI.

Table 8.1: Test Plan

Test Area	What Was Tested	Tool / Method	Timing	Evidence
Authentication and user accounts	Provider registration, respondent registration, duplicate email rejection, and inactive account login rejection.	pytest service tests	During authentication feature work and final verification.	tests/services/test_authentication_service.py; local pytest passed.
Survey lifecycle	Create, update, delete, publish, close, ownership checks, and status rules.	pytest service tests	During survey module work and final verification.	tests/services/test_survey_service.py; local pytest passed.
Questions and targeting	Question types, options, question ordering, targeting criteria validation, and demographic matching.	pytest service tests	During survey preparation feature work and final verification.	tests/services/test_question_service.py; tests/services/test_targeting_service.py; local pytest passed.
Respondent eligibility	Eligible published unanswered surveys for respondent profiles.	pytest service tests	During respondent workflow work and final verification.	tests/services/test_survey_service.py; tests/services/test_targeting_service.py; local pytest passed.
Response submission	Valid response storage, answer validation, and restrictions for draft or closed surveys.	pytest service tests	During response workflow work and final verification.	tests/services/test_response_service.py; local pytest passed.
Duplicate prevention	Rejecting a second response to the same survey by the same respondent.	pytest service tests and database constraint inspection	During response workflow work and final verification.	tests/services/test_response_service.py; apps/core/models/response.py; local pytest passed.

Sentiment analysis	Open-text analysis, non-text answer exclusion, failed external analysis handling, and analytics summaries.	pytest service tests with test doubles	During analysis module work and final verification.	tests/services/test_analysis_service.py; services/analysis_service.py; local pytest passed.
Admin functions	Activate, suspend, soft-delete accounts, self-action restrictions, and system reports.	pytest service tests	During admin module work and final verification.	tests/services/test_admin_service.py; local pytest passed.
UI workflow validation	Forms, navigation, dashboards, messages, and role-specific pages.	Manual browser validation	Before final delivery.	UI screenshots should be inserted as final evidence.
CI validation	Django system check and service test command in GitHub Actions.	GitHub Actions workflow	After pushing workflow to GitHub.	GitHub Actions CI completed successfully.

8.2 Testing Types

Table 8.2: Testing Types

Testing Type	Applied?	Description	Evidence / Status
Unit testing	Limited	Individual service methods and validation rules are tested through service test files.	Covered through service-level pytest tests; the suite passed locally.
Service-layer testing	Yes	Business workflows are tested at the service layer.	163 tests passed using python -m pytest tests/services -q.
Integration testing	Limited	Tests use Django settings and database-backed models through pytest-django.	pytest.ini, requirements.txt, and service tests.
System testing	Limited	Full browser end-to-end testing is not evidenced as an automated suite.	Manual workflow screenshots are recommended.
Acceptance testing	Not formally verified	No signed acceptance checklist or user acceptance evidence was found.	Can be added through supervisor or stakeholder review.
Performance testing	Not formally verified	No benchmark or measured response-time output exists.	Response time remains a target, not a verified metric.

Security testing	Limited	Configuration and code inspection support authentication, CSRF protection, role checks, ownership checks, and secret isolation.	No penetration testing evidence exists.
Usability testing	Limited	Internal/manual review can be described, but no real user survey/interview evidence exists.	UI screenshots and user feedback can strengthen this section.
Accessibility testing	Not formally verified	No accessibility testing tool output or checklist evidence exists.	Do not claim accessibility compliance.
Compatibility testing	Not formally verified	No cross-browser or device test matrix evidence exists.	Screenshots and browser checks would be required.

8.3 Test Diagrams

Test diagrams and screenshots are used to summarize selected testing coverage and test result evidence. Figure 8.1 summarizes the service-layer test coverage groups. Figures 8.2 and 8.3 provide execution evidence for local pytest execution and GitHub Actions CI.

8.3.1 Test Case Diagrams

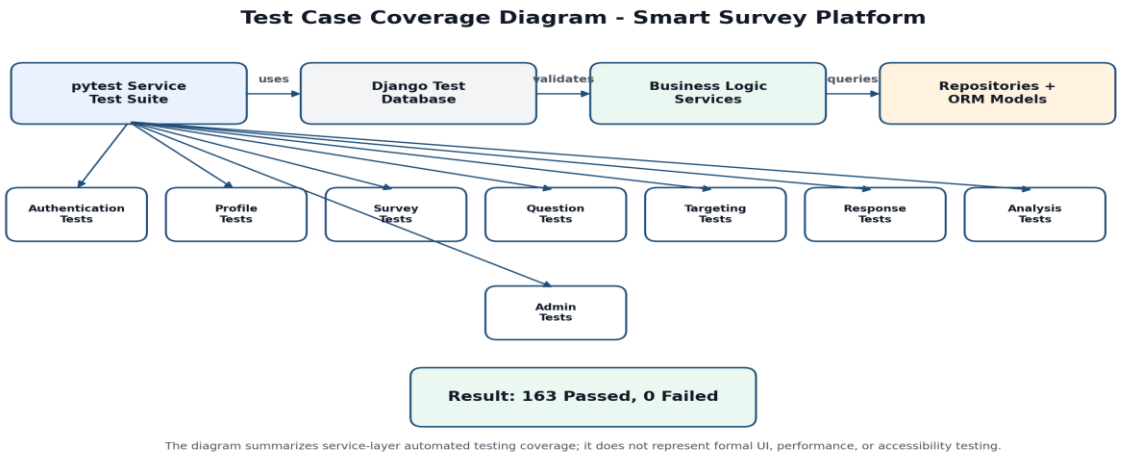

Figure 8.1: Test Case Coverage Diagram

8.3.2 Test Result Diagrams

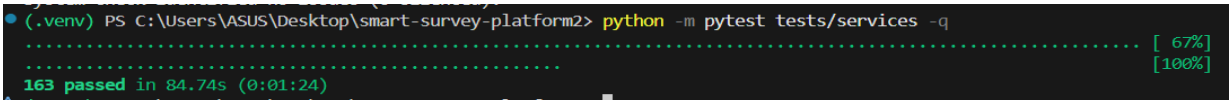

Figure 8.2: Pytest Execution Result Screenshot

The local service-layer test suite was executed successfully using pytest. The result shows 163 passed tests, 0 failed tests, and total execution time of 84.74 seconds.

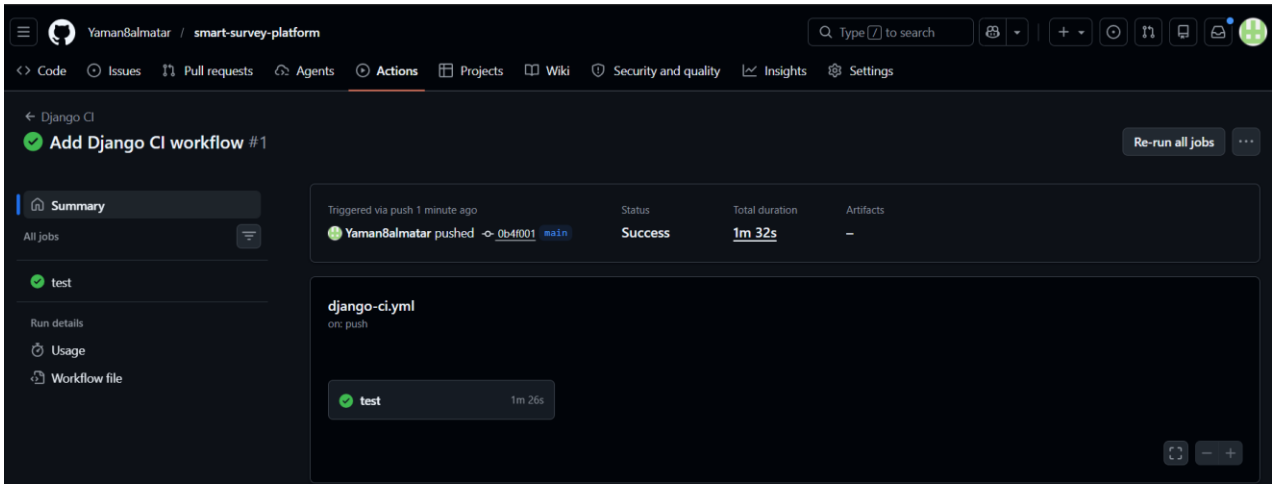

Figure 8.3: GitHub Actions CI Result Screenshot

The GitHub Actions CI workflow executed successfully after pushing the repository changes. The workflow completed with status Success and total duration of 1 minute and 32 seconds.

8.4 Test Results and Analysis

The automated service-layer test suite was executed successfully. The local pytest result reported 163 passed tests and 0 failed tests. The GitHub Actions CI workflow also completed successfully. These results verify the main automated service-layer workflows, while UI, performance, accessibility, and formal user testing remain limited or not formally verified.

Table 8.3: Test Results Summary

Test Category	Test Cases / Evidence	Passed	Failed	Status	Notes
Authentication service	Authentication-related service tests.	Passed	0	Passed	Covers registration, duplicate email, login, and inactive account rejection.
Profile service	Profile service tests.	Passed	0	Passed	Covers respondent profile retrieval, update, and validation rules.
Survey service	Survey lifecycle tests.	Passed	0	Passed	Covers create, update, delete, publish, close, ownership, and eligibility behavior.
Question service	Question management tests.	Passed	0	Passed	Covers question types, options, ordering, update, and delete service behavior.
Targeting	Targeting	Passed	0	Passed	Covers targeting save/update and

service	criteria and demographic matching tests.				respondent-to-survey matching.
Response service	Response submission tests.	Passed	0	Passed	Covers response storage, duplicate prevention, answered surveys, and answer validation.
Analysis service	Sentiment analysis and analytics tests.	Passed	0	Passed	Covers API failure behavior, label mapping, and analytics summaries.
Admin service	Administration tests.	Passed	0	Passed	Covers account actions and system report generation.
Django system check	GitHub Actions runs python manage.py check.	Passed	0	Passed	CI workflow completed successfully.
CI service tests	GitHub Actions runs python -m pytest tests/services -q.	Passed	0	Passed	CI workflow completed successfully.

Table 8.4: Main Test Case Results

Test Case	Scenario	Expected Result	Actual Result	Status	Related Use Case
TC-01	Register service provider.	A service provider account is created with valid data.	Passed in service-layer pytest execution.	Passed	UC-01
TC-02	Register respondent.	A respondent account and demographic profile are created.	Passed in service-layer pytest execution.	Passed	UC-02
TC-03	Reject duplicate email.	Registration with an existing email is rejected.	Passed in service-layer pytest execution.	Passed	UC-01, UC-02
TC-04	Reject inactive account login.	Suspended or deleted accounts cannot authenticate.	Passed in service-layer pytest execution.	Passed	UC-03
TC-05	Add supported question type.	Supported question data is saved for an owned draft survey.	Passed in service-layer pytest execution.	Passed	UC-08
TC-06	Publish incomplete	A survey without required publication conditions is	Passed in service-layer pytest	Passed	UC-11

	survey rejection.	rejected.	execution.		
TC-07	Filter eligible surveys.	Respondent sees only published, matching, unanswered surveys.	Passed in service-layer pytest execution.	Passed	UC-15
TC-08	Reject duplicate response.	A second response to the same survey by the same respondent is rejected.	Passed in service-layer pytest execution.	Passed	UC-16
TC-09	Analyze open-text answer safely.	Open-text answers are analyzed, and failed analysis creates a failed result without deleting the response.	Passed in service-layer pytest execution.	Passed	UC-16, UC-13
TC-10	Admin suspends account.	An administrator can suspend another user account but cannot suspend self.	Passed in service-layer pytest execution.	Passed	UC-18

8.5 Quality Metrics

Table 8.5: Quality Metrics

Metric	Value / Status	Interpretation
Number of service test files	8	Detected under tests/services/.
Number of passed service tests	163 passed	Verified by local pytest execution.
Failed tests	0	No failed tests were reported in the pytest output.
Local test execution time	84.74 seconds	The service-layer test suite completed in approximately 1 minute and 24 seconds.
CI status	Passed	GitHub Actions completed successfully with total duration of 1 minute and 32 seconds.
Coverage percentage	Not measured	No coverage report was generated.
Defect density	Not calculated	No formal defect log exists.
Response time	Target only	No page response-time benchmark was performed.
Known remaining defects	Not formally logged	No defect log exists. UC-09 question update/delete remains partially implemented at the web UI level if not completed later.

Section Five: Results and Recommendations

Chapter 9: Results and Analysis

The purpose of this chapter is to present the actual outputs produced by the Smart Survey Platform and evaluate the system against the project objectives. The evaluation focuses on system outputs, comparison with existing systems, objective achievement, strengths and weaknesses, and measurable performance indicators.

9.1 System Implementation Results

The implemented system produced working outputs for the three main user roles: service provider, respondent, and system administrator. The outputs include role-based dashboards, survey management pages, respondent eligibility filtering, response submission pages, analytics dashboards, administration pages, and testing/CI evidence.

Table 9.1: Actual System Outputs

Area	System Output
Public Interface	Public home page, login page, and registration pages for service providers and respondents.
Service Provider	Provider dashboard, survey list, create/edit/delete survey pages, question management, targeting criteria, publish/close actions, and analytics dashboard.
Respondent	Respondent dashboard, demographic profile page, eligible surveys page, submit response page, confirmation page, and answered surveys page.
Administrator	Admin dashboard, user management table, account status actions, and system reports page.
AI Analysis	Sentiment analysis records for open-text answers with status, label, score, and analysis timestamp.
Database Output	Stored users, profiles, surveys, questions, options, targeting criteria, responses, answers, and sentiment analysis results.
Testing Output	Local pytest execution result showing 163 passed tests and 0 failed tests.
CI Output	GitHub Actions CI workflow completed successfully with status Success.

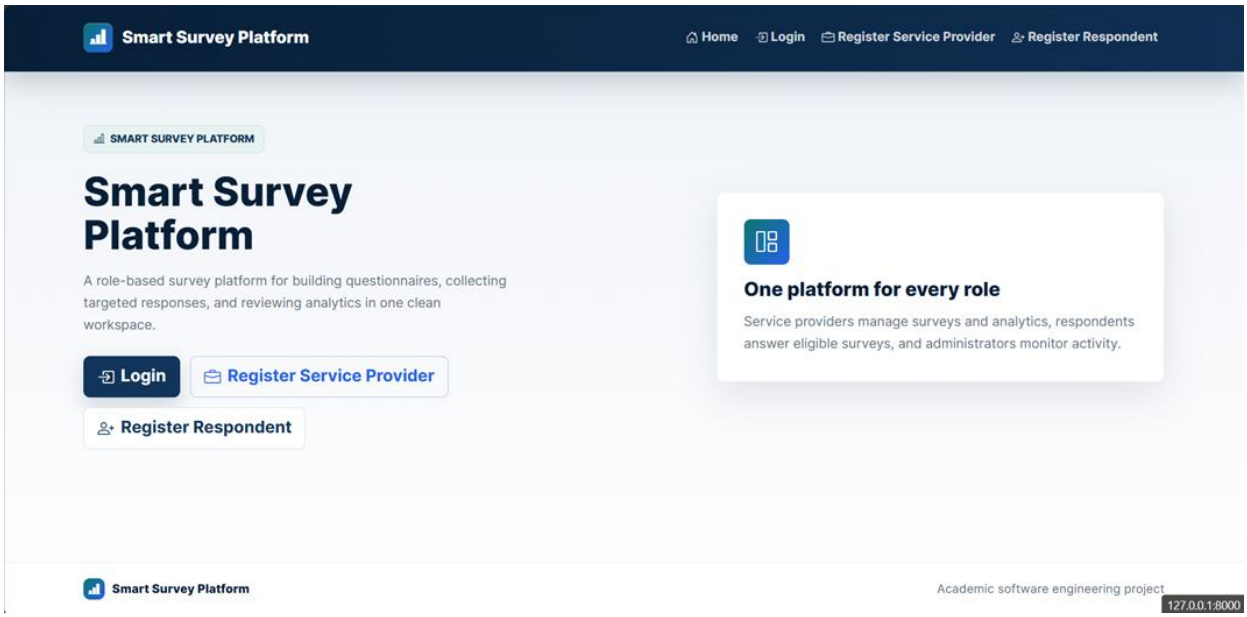

Figure 9.1: Public Home Page

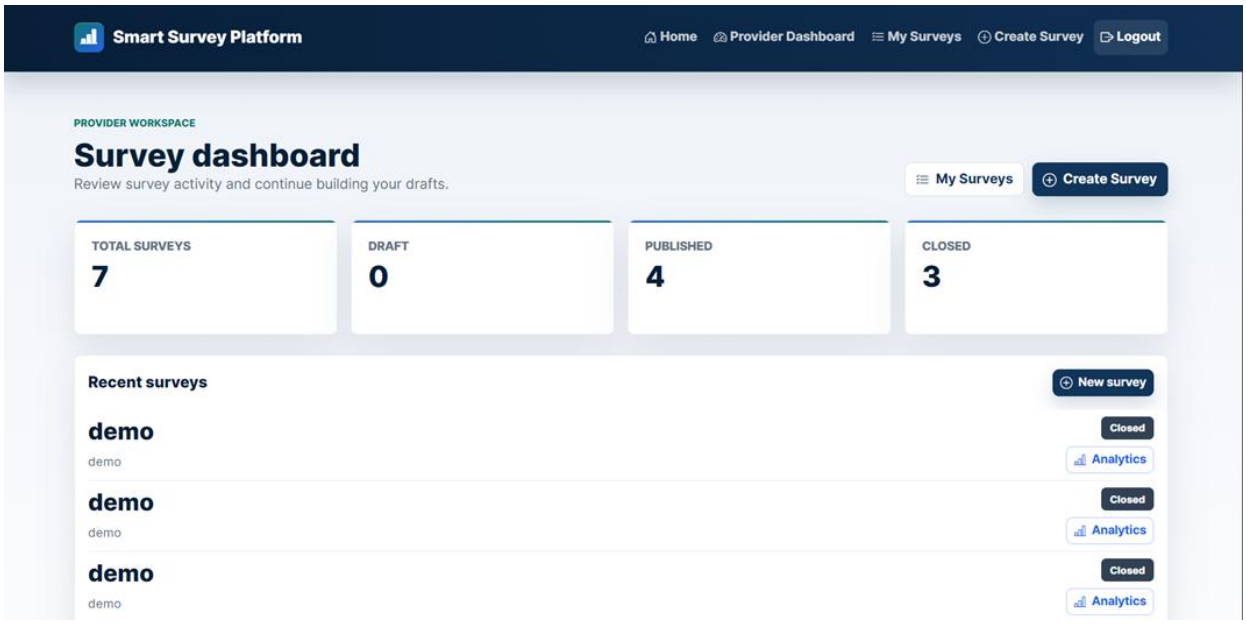

Figure 9.2: Provider Dashboard

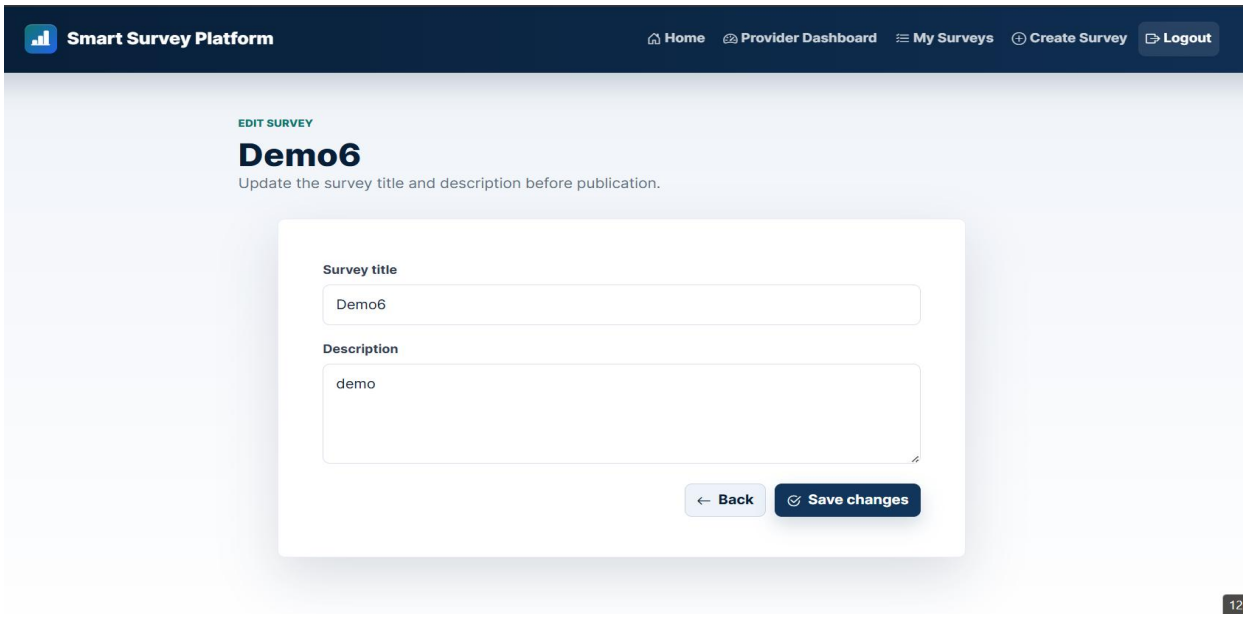

Figure 9.3: Survey Edit Page

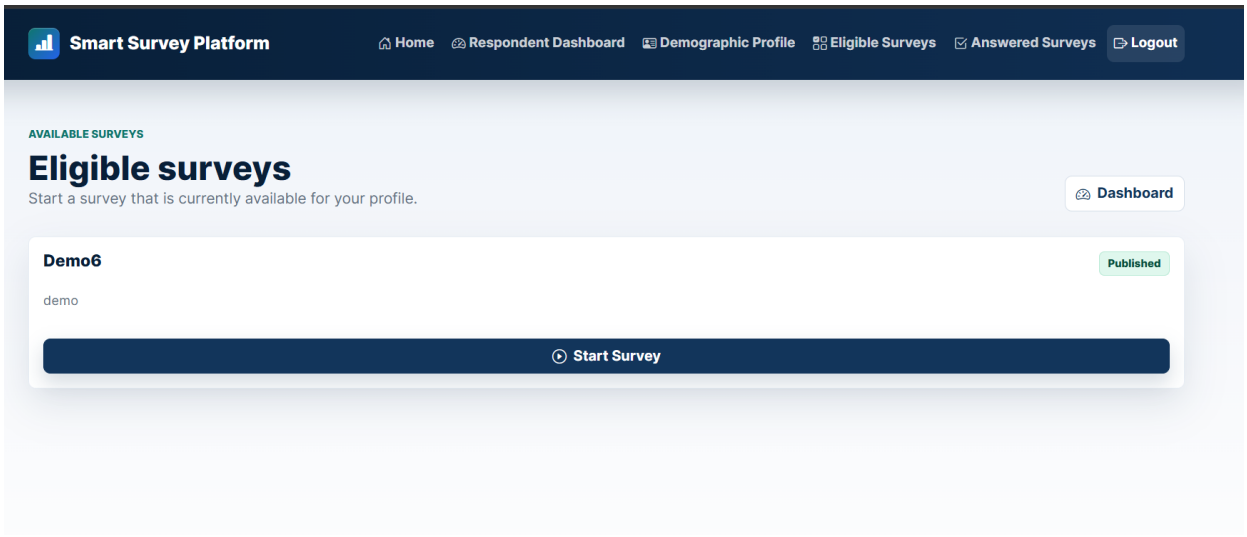

Figure 9.4: Eligible Surveys Page

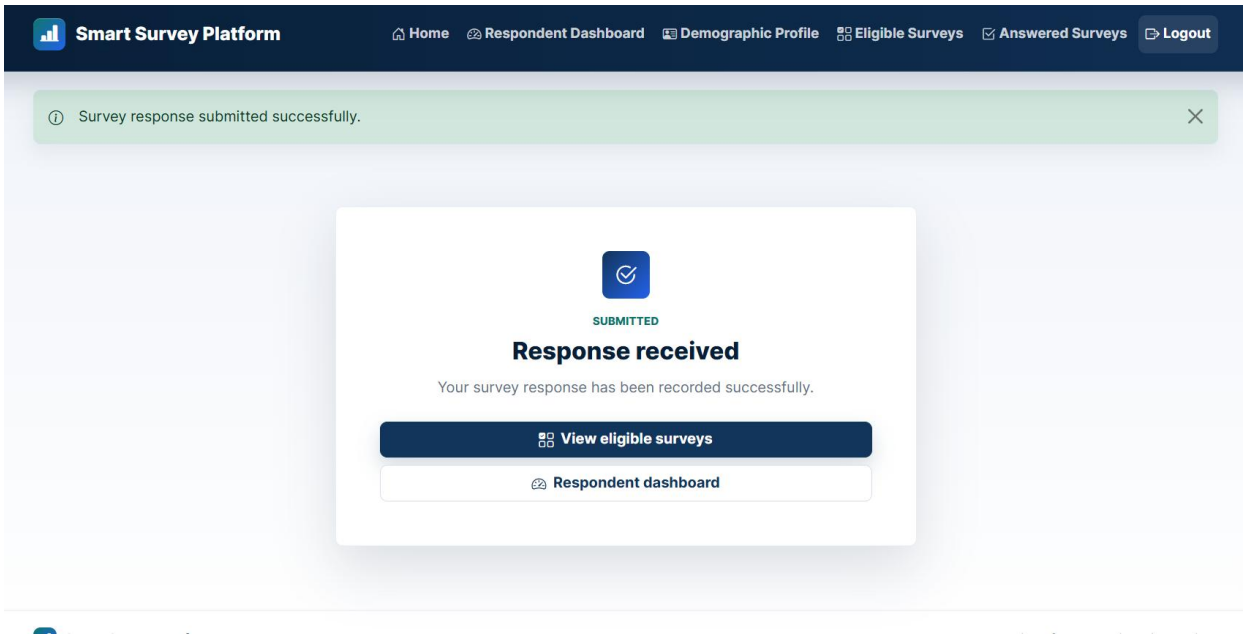

Figure 9.5: Submission Confirmation Page

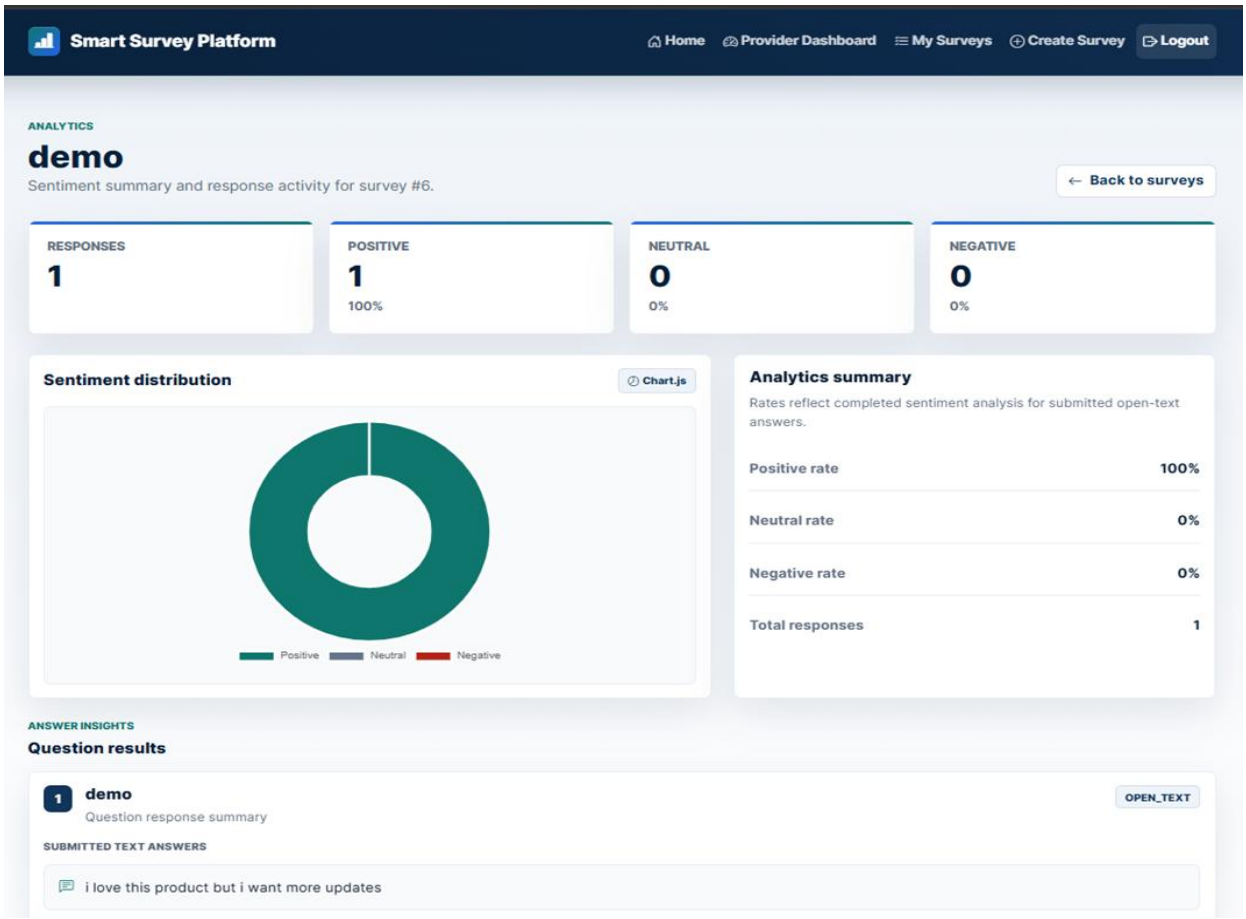

Figure 9.6: Analytics Dashboard

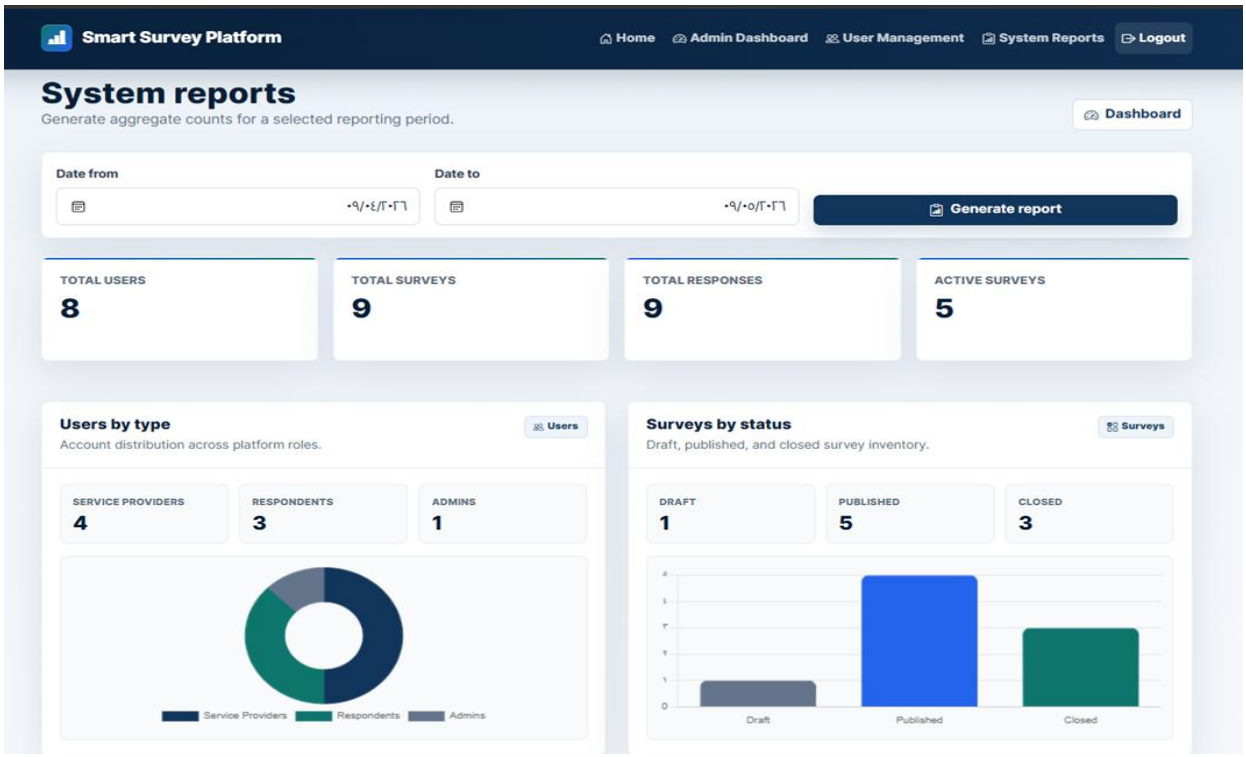

Figure 9.7: Admin System Reports Page

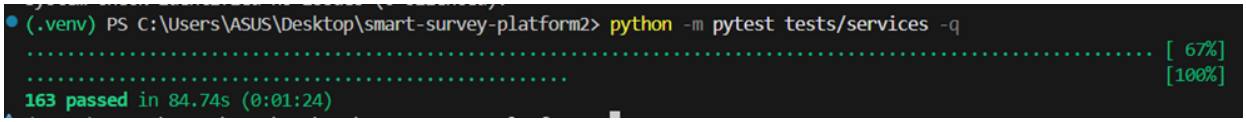

Figure 9.8: Local Pytest Execution Result

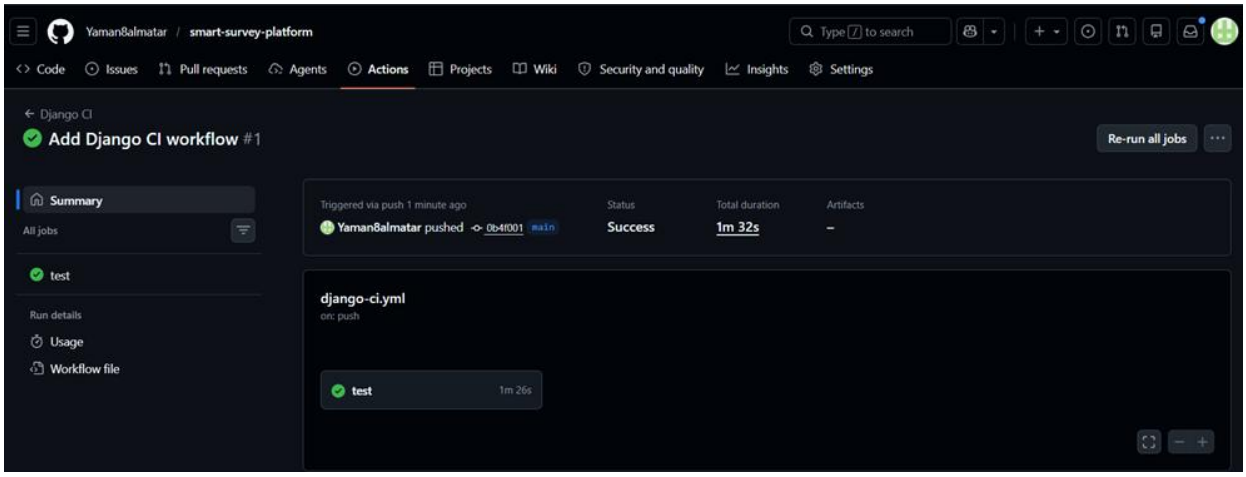

Figure 9.9: GitHub Actions CI Result

9.2 Comparative Analysis with Existing Systems

The Smart Survey Platform was compared with Google Forms, SurveyMonkey Audience, and Qualtrics. Google Forms is suitable for online forms and surveys with real-time analysis. SurveyMonkey Audience supports buying responses and targeting respondent groups by attributes such as country, gender, age range, income, and other options. Qualtrics supports advanced research workflows, customer research panels, and quality controls. The comparison below focuses on speed, data quality, cost, targeting, AI analysis, duplicate prevention, and implementation transparency.

Table 9.2: Comparative Analysis with Existing Systems

Criterion	Google Forms	SurveyMonkey Audience	Qualtrics	Smart Survey Platform
Speed	Fast for creating and sharing basic surveys.	Fast respondent acquisition through paid audience services.	Fast for enterprise research workflows.	Fast for academic/local workflows; formal page response-time benchmarking was not performed.
Accuracy / Data Quality	Basic response collection; targeting is mostly controlled through sharing and account settings.	Provides targeted respondents and platform-managed audience workflows.	Provides research-oriented audience and data-quality controls.	Uses profile-based eligibility filtering and duplicate response prevention.
Cost	Low cost for basic usage; advanced usage depends on Google Workspace plan.	Paid audience acquisition model.	Enterprise-oriented commercial platform.	Low academic implementation cost using open-source frameworks and an external AI API.
Targeting	Limited in the basic form workflow.	Supports targeting by demographic and other attributes.	Supports panels and advanced research audience management.	Built-in targeting by gender, age range, and region.
AI Analysis	Not the main standard workflow of basic forms.	Depends on product plan and ecosystem.	Advanced analytics ecosystem for enterprise research.	Integrated sentiment analysis for open-text answers.
Duplicate Prevention	Limited/configurable depending on settings.	Controlled through platform workflows.	Controlled through research platform workflows.	Enforced by service logic and database unique constraint.
Transparency	Commercial platform; internal implementation is not visible.	Commercial platform; internal implementation is not visible.	Commercial platform; internal implementation is not visible.	Fully documented academic implementation with requirements, UML, services, repositories, tests, and CI.

Source note: the comparison uses official product descriptions for Google Forms, SurveyMonkey Audience, and Qualtrics. The Smart Survey Platform column is based on the implemented project outputs and testing evidence.

9.3 Achievement of Objectives

This section evaluates the implemented system against the objectives defined in Chapter 1. The evaluation is based on the actual system behavior, service-layer tests, CI result, and documented implementation evidence.

Table 9.3: Project Objectives Achievement

Objective	Evaluation	Status
Build a web-based smart survey platform.	The system was implemented as a Django web application with role-based pages and dashboards.	Achieved
Allow service providers to manage surveys.	Providers can create, update, delete draft surveys, publish complete surveys, close published surveys, and view analytics.	Achieved
Support multiple question types.	The system supports multiple-choice, rating-scale, and open-text questions.	Achieved
Support demographic targeting.	Targeting criteria are implemented using gender, age range, and region.	Achieved
Show only eligible surveys to respondents.	Eligibility filtering is implemented using respondent profile data and targeting criteria.	Achieved
Prevent duplicate participation.	Duplicate responses are prevented through service validation and a database unique constraint.	Achieved
Analyze open-text answers intelligently.	Open-text answers are analyzed using the Hugging Face Inference API and stored as sentiment results.	Achieved
Provide analytics and reports.	Provider analytics and administrator reports are implemented.	Achieved
Apply software engineering traceability.	Requirements, use cases, architecture, UML, tests, and CI evidence are documented.	Achieved
Perform production deployment.	Production deployment was not part of the current academic scope.	Not within scope
Perform formal performance, accessibility, and user satisfaction studies.	These studies were not formally conducted with measurement tools or real user surveys.	Not formally verified

9.4 Strengths and Weaknesses Analysis

The system meets the main functional and academic objectives, but the evaluation must remain realistic. The strongest areas are architecture, traceability, testing, role separation, targeting, duplicate prevention, and AI-assisted sentiment analysis. The weaker areas are formal performance evaluation, accessibility testing, model accuracy measurement, and real-user usability testing.

Table 9.4: Strengths and Weaknesses

Area	Strengths	Weaknesses / Limitations
Architecture	Clear layered architecture: presentation, services, repositories, domain models, and infrastructure.	Further refinements could move more UI preparation logic into dedicated helpers or service methods.
Functional Scope	Core workflows for providers, respondents, and administrators are implemented.	Question update/delete should be confirmed as a complete web UI workflow before claiming full UI completion.
Testing	Automated service-layer tests passed: 163 passed and 0 failed.	Test coverage percentage was not measured.
CI	GitHub Actions CI completed successfully.	Continuous deployment is not implemented.
Data Integrity	Duplicate response prevention is enforced by service logic and database constraint.	No formal stress testing was conducted on large datasets.
AI Analysis	Open-text answers are analyzed automatically through an external sentiment model.	Model accuracy was not measured using a labeled project-specific dataset.
Reliability	If the external AI call fails, the submitted response remains saved and analysis status becomes FAILED.	No asynchronous retry queue is implemented.
UI	Role-specific dashboards and pages are implemented using Django templates and Bootstrap.	Formal usability testing with real users was not conducted.
Security	Password hashing, CSRF, role checks, ownership checks, and environment variables are used.	No penetration testing or formal security audit was conducted.
Accessibility	Bootstrap provides a responsive base.	No formal accessibility audit was conducted.

9.5 Key Performance Indicators (KPIs)

The following KPIs summarize the measurable results of the implemented system. Only verified values are reported. Unmeasured indicators are marked clearly to avoid unsupported claims.

Table 9.5: System KPIs

KPI	Result / Status	Interpretation
Automated service tests	163 passed	Verified by local pytest execution.
Failed automated tests	0 failed	No failed service tests were reported.
Pytest execution time	84.74 seconds	The service-layer test suite completed in approximately 1 minute and 24 seconds.
CI status	Success	GitHub Actions CI completed successfully.

CI duration	1 minute 32 seconds	The CI workflow executed Django check and service tests successfully.
Test coverage percentage	Not measured	No coverage report was generated.
Model accuracy	Not measured	No labeled dataset was used to calculate accuracy, precision, recall, or F1-score.
User satisfaction	Not measured	No formal user survey or interview was conducted.
Page response time	Not formally measured	No browser timing or load testing evidence was collected.
Defect density	Not calculated	No formal defect log was maintained.
Cost indicator	Low academic implementation cost	The system uses open-source frameworks and an external API rather than commercial respondent acquisition or enterprise platform licensing.

Overall, the Smart Survey Platform achieved its main academic and functional objectives. The strongest evidence is the successful implementation of the main workflows, the passing service-layer test suite, and the successful GitHub Actions CI workflow. However, model accuracy, usability satisfaction, accessibility compliance, and formal response-time performance remain future evaluation areas because they were not measured using dedicated tools or real-user studies.

Chapter 10: Conclusion and Recommendations

The purpose of this chapter is to close the project report by summarizing the main achievements of the Smart Survey Platform, identifying the challenges faced during development, proposing future improvements, and presenting recommendations for future students or organizations that may extend or adopt the system.

10.1 Project Summary and Main Achievements

The Smart Survey Platform was developed as a web-based system for creating, managing, targeting, answering, and analyzing surveys. The system was implemented using Django and follows a layered architecture that separates presentation, business logic, data access, domain models, and external AI integration.

The project achieved its main objective by delivering a functional survey platform with three main user roles: service provider, respondent, and system administrator. Service providers can manage surveys, questions, targeting criteria, publication status, and analytics. Respondents can manage demographic profiles, view eligible surveys, submit responses, and view answered surveys. Administrators can manage user accounts and generate system reports.

A major achievement of the project is the integration of AI-assisted sentiment analysis for open-text answers using the Hugging Face Inference API. This adds an intelligent component to the platform and allows qualitative feedback to be summarized using sentiment labels and scores.

The project also achieved strong engineering traceability. Requirements were documented, use cases were defined, UML and ERD assets were prepared, the system architecture was designed, and the implementation was validated through service-layer tests and a GitHub Actions CI workflow. The automated service-layer test suite passed successfully with 163 passed tests and 0 failed tests, and the CI workflow completed successfully.

Table 10.1: Main Project Achievements

Achievement	Description
Web-based survey platform	Implemented as a Django server-rendered web application.
Role-based system	Supports service provider, respondent, and system administrator roles.
Survey management	Supports survey creation, update, deletion, publishing, closing, and analytics.
Question management	Supports multiple-choice, rating-scale, and open-text question types.
Targeting criteria	Supports demographic targeting by gender, age range, and region.
Response submission	Allows respondents to answer eligible surveys.

Achievement	Description
Duplicate prevention	Prevents repeated participation using service validation and database constraints.
Smart analysis	Performs sentiment analysis for open-text answers using Hugging Face API.
Administration	Supports account status management and system report generation.
Testing	Service-layer test suite passed with 163 passed tests and 0 failed tests.
CI workflow	GitHub Actions CI workflow completed successfully.

10.2 Difficulties and Challenges Faced by the Project

Several challenges were faced during the development and documentation of the project. These challenges were technical, organizational, and time-related.

From the technical side, one challenge was maintaining a clean layered architecture. Django projects can easily mix views, business logic, and database operations if no strict structure is applied. To avoid this, the project separated the code into views, services, repositories, models, and infrastructure clients.

Another challenge was integrating the external Hugging Face API safely. External API calls can fail due to missing tokens, invalid responses, network issues, or service unavailability. The system handled this by saving the respondent response first, then marking sentiment analysis as failed if the external API call fails.

Testing was also a challenge. The project required a valid Python environment and PostgreSQL configuration. The local environment initially had incompatible Python execution issues, but this was addressed by running tests inside the correct virtual environment and by configuring GitHub Actions CI.

Documentation and diagram consistency were also difficult because the project evolved from earlier naming and design assets. Some old materials used previous names or incomplete diagrams. These assets required auditing and correction to match the final implementation.

Table 10.2: Project Challenges

Challenge Type	Challenge	Handling / Mitigation
Technical	Maintaining separation between views, services, repositories, and models.	Applied layered architecture and repository pattern.
Technical	External AI API failure risk.	Saved responses before analysis and marked failed analysis safely.
Technical	Preventing duplicate responses.	Used service-level validation and database unique constraint.
Technical	Keeping diagrams aligned with actual code.	Audited and corrected UML, ERD, and sequence diagrams.
Environment	Python environment incompatibility during testing.	Used correct virtual environment and configured GitHub Actions CI.
Documentation	Old project naming and legacy assets existed.	Standardized current name as Smart Survey Platform.
Time	Preparing implementation, testing, diagrams, screenshots, and report together.	Organized outputs by chapters and evidence-based documentation.

10.3 Future Development Suggestions

Although the current system satisfies the main academic and functional objectives, several improvements can be added in future work if the project continues.

First, the system can be extended with a full public REST API to support mobile applications or third-party integrations. This should be done only after defining API authentication, versioning, validation, and documentation.

Second, the AI analysis can be improved by evaluating sentiment model accuracy using a labeled dataset. The current system uses an external pretrained model through an API, but it does not measure model accuracy, precision, recall, or F1-score on project-specific data.

Third, performance testing can be added using benchmark tools and larger datasets. This would provide actual response-time metrics instead of only a target value.

Fourth, usability testing with real users can be conducted through questionnaires or structured interviews. This would provide measurable user satisfaction and interface feedback.

Fifth, accessibility testing should be added using tools such as Lighthouse or WCAG checklists. This would improve the system for users with different accessibility needs.

Sixth, the system can be improved by adding asynchronous background processing for sentiment analysis. This would prevent external API latency from affecting the response submission workflow.

Table 10.3: Future Development Suggestions

Suggested Improvement	Expected Benefit
Public REST API	Enables mobile applications and third-party integration.
Mobile application	Allows respondents to participate from mobile devices more conveniently.
Advanced AI evaluation	Measures model accuracy, precision, recall, and F1-score using labeled data.
Asynchronous AI jobs	Improves reliability and avoids waiting for external API calls during submission.
Retry queue for failed analysis	Allows failed sentiment analysis tasks to be retried later.
Formal performance testing	Provides measured response-time and load-handling evidence.
Accessibility audit	Improves usability for users with accessibility requirements.
Real-user usability testing	Measures satisfaction and identifies user interface weaknesses.
Exportable reports	Allows administrators and service providers to export analytics as PDF or Excel.
Notification system	Notifies users about survey status, new eligible surveys, or analysis completion.
Production deployment	Allows the system to be accessed outside the local development environment.

10.4 Recommendations

Several recommendations can be made for future students, developers, or organizations that may continue or adopt the Smart Survey Platform.

Future students should begin with requirements analysis and system design before implementation. Starting with code before defining requirements, use cases, architecture, and database design increases the risk of inconsistent implementation.

Developers extending the project should preserve the layered architecture. Views should remain responsible for HTTP behavior and rendering only, while business rules should remain inside

services, database operations inside repositories, and external API calls inside infrastructure clients.

Organizations adopting the system should conduct additional evaluation before production use. This includes performance testing, accessibility testing, security review, user acceptance testing, and deployment planning.

The system should also avoid storing real API tokens or secrets in source code. Environment variables and ignored .env files should continue to be used for sensitive configuration.

Finally, future work should focus on measurable improvement. Claims about performance, user satisfaction, AI accuracy, or accessibility should only be made when supported by actual testing evidence.

Table 10.4: Recommendations

Target Group	Recommendation
Future students	Start with requirements, use cases, architecture, and database design before coding.
Future developers	Preserve separation of concerns between views, services, repositories, models, and infrastructure.
Future developers	Add new features through service and repository classes rather than placing business logic in views.
AI developers	Evaluate sentiment analysis accuracy using a labeled dataset before claiming model performance.
Testing team	Maintain automated tests and expand them with coverage reporting and UI/system tests.
UI team	Conduct usability testing with real users and improve the interface based on feedback.
Security reviewers	Perform a formal security review before any production deployment.
Organizations	Perform performance, accessibility, and acceptance testing before adopting the system operationally.
Deployment team	Add production deployment only after defining hosting, security, database backup, and monitoring plans.

In conclusion, the Smart Survey Platform achieved its main project goals as an academic software engineering system. It demonstrates a working implementation of survey creation, respondent targeting, response submission, AI-assisted sentiment analysis, dashboards, administration, automated testing, and CI validation. The project is technically acceptable for its current scope, while future work should focus on formal performance evaluation, usability testing, accessibility compliance, production deployment, and deeper AI model evaluation.